Destiny Church Tees Valley — Complete Repository Documentation

Version: 1.0.8
Last Updated: August 8, 2026
Repository: Square Media Group — destinychurch

This document provides a comprehensive explanation of every major component, line of code purpose, architecture decisions, and how the system works from end-to-end.

Table of Contents

Project Overview

What It Is

Destiny Church Tees Valley is a **full-stack church website platform** built by Square Media Group. It serves as the digital presence for a multi-cultural church in Stockton-on-Tees, UK, and handles:

- **Public Pages:** About, beliefs, events, contact, giving, venue hire
- **Ministry Pages:** Kids, youth, young adults, missions, connect groups
- **Media:** Sermon archive with 50+ videos, series/speaker filtering, AI-powered search
- **Member Engagement:** Prayer requests, volunteer signup, venue enquiries, connection forms
- **Admin Dashboard:** Protected area for staff to manage sermons, pages, redirects, banners, HR, jobs, training
- **Infrastructure:** Email, analytics, webhooks, caching, rate limiting

Key Visitors

- **Church members:** Watch sermons, find events, volunteer, connect groups, training
- **First-time visitors:** Plan a visit, explore beliefs, understand what to expect
- **HR/Admin staff:** Manage jobs, staff directory, leave requests, documents
- **Job applicants:** Browse positions, submit applications
- **Training participants:** Access course materials and learning paths

Non-Goals

- Real-time chat or messaging
- Video hosting (uses YouTube)
- Full ecommerce (giving is simplified)
- Social network features

Architecture & Philosophy

Key Design Principles

1. **Server-Driven:** Use server-side rendering and server actions wherever possible; minimize client-side complexity
2. **Security First:** Row-level security (RLS) on all sensitive data; rate limiting on public APIs; Supabase service role for backend operations
3. **Edge Caching:** Long TTLs on static assets; revalidate on-demand for dynamic content
4. **Mobile First:** Responsive Tailwind CSS; tested on real devices
5. **Accessibility:** Semantic HTML, ARIA labels, keyboard navigation
6. **AI-Ready:** Structured data for future AI features (smart search, content generation)
7. **No Vendor Lock-In:** Open-source stack (Next.js, React, Tailwind, Supabase PostgreSQL)

Data Flow

```
User (Browser/Phone)
  ↓
Next.js App Router (Server/Client)
  ↓
API Routes (Node.js Serverless)
  ↓
Supabase PostgreSQL
  ↓
Responses: JSON, Server-Side HTML, Redirects, Cached Assets
```

Authentication Layers

- **Public Pages:** No auth required; cached at edge
- **Member Features:** Supabase Auth (email/password); tokens in secure cookies
- **Admin Dashboard:** Supabase Auth required; additional role checks via proxy

- **API Endpoints:** Service role key (server-to-database); user tokens (client-to-server)

Technology Stack

Layer	Technology	Purpose
Framework	Next.js 16 (App Router)	Full-stack React with server-side rendering
Language	TypeScript	Type-safe code
Styling	Tailwind CSS v4	Utility-first CSS framework
Frontend	React 19	Component library and state management
Database	Supabase (PostgreSQL)	Relational database with RLS and real-time
Auth	Supabase Auth	User sessions and JWT tokens
Email	Resend	Transactional email (password resets, confirmations)
Storage	Supabase Storage	File uploads (HR documents, media, images)
Video	YouTube API v3	Sermon hosting and metadata
Podcasts	Buzzsprout	Podcast RSS and metadata
AI	OpenAI (`gpt-4.1-mini`)	Smart Search tool-calling chat (products, weather, directions, web search, page extraction)
Web Search	Tavily (`search` + `extract` APIs)	Smart Search's `search_web` / `extract_page` tools — live facts, page content
Bot Protection	Cloudflare Turnstile	Gates `/login` sign-in and the Smart Search `/api/chat` endpoint
Payments	Stripe (Payment Element + Express Checkout)	`/shop` checkout — cards, Apple Pay, Google Pay, Link
Analytics	Vercel Analytics + SpeedInsights	Performance and visitor tracking
Deployment	Vercel	Edge functions, serverless, CDN
Testing	Playwright	E2E browser testing
Media Processing	Sharp	Uploaded images (posts, shop products, shop hero) resized/converted to WebP server-side; `ffmpeg-static` bundled but unused
Search	fuse.js + rbush	Client-side fuzzy search and spatial indexing
Icons	Phosphor + Material Symbols	Icon sets
Rich Text	TipTap (Prosemirror)	Sermon markdown, content editing
State	Zustand	Global client state (minimal usage)
Motion	Motion	Animation library
QR Codes	qrcode.react	QR code generation

Directory Structure

```
destinychurch/
├── app/                            # Next.js App Router routes
│   ├── layout.tsx                  # Root layout (header, footer, providers)
│   ├── page.tsx                    # Home page
│   ├── globals.css                 # Tailwind + custom CSS
│   ├── robots.ts                   # robots.txt generation
│   │                                # NOTE: there is no `(public-pages)` route group — every
│   │                                # page below is a flat top-level folder directly under app/
│   ├── about/                      # About page
│   ├── accessibility/              # Reduced-motion / glass-FX preferences (client component)
│   ├── admin-login/                 # Stale-bookmark redirect → /login
│   ├── administration/             # Stale-bookmark redirect → /admin
│   ├── auth/                       # callback/ + confirm/ — Supabase OAuth & email-OTP handlers
│   ├── baptism/                    # Baptism sign-up
│   ├── beliefs/                    # Beliefs page
│   ├── bible-course/               # The Bible Course (Bible Society) info page
│   ├── cap-money/                  # CAP Money Course (Christians Against Poverty) info page
│   ├── child-dedication/           # Child dedication request
│   ├── connect/                    # Connect groups page
│   ├── data-gdpr/                  # Data & GDPR policy
│   ├── dckids/                     # Destiny Kids Camp 2026 campaign page
│   ├── destiny-recovery/           # Recovery course info page
│   ├── give/                       # Giving/donations page
│   ├── help/                       # Help centre / FAQ
│   ├── kids/                       # Kids ministry
│   ├── links/                      # "Next Steps" link-in-bio style page
│   ├── live/                       # Livestream page
│   ├── login/                      # Staff sign-in
│   ├── youth/                      # Youth ministry
│   ├── young-adults/               # Young adults ministry
│   ├── missions/                   # Missions & outreach
│   ├── privacy/                    # Privacy policy
│   ├── governance/                # Charity & company registration / transparency page
│   ├── safeguarding/               # Safeguarding policy
│   ├── serve/                      # Volunteer opportunities (overview)
│   ├── sermons/                    # Sermon archive (+ [id] detail)
│   ├── terms/                      # Terms of use
│   ├── training/                   # /training resource library (category → subgroup → post)
│   ├── contact/                    # Contact form
│   ├── visit/                      # Plan a visit
│   ├── new-here/                   # First-time visitor guide
│   ├── hire/                       # Venue hire enquiries
│   ├── connect-card/               # Prayer requests & connections
│   ├── alpha/                      # Alpha course info
│   ├── volunteer/                  # Volunteer sign-up form
│   ├── whats-on/                   # Events listing + per-event pages ([slug])
│   ├── [slug]/                     # Dynamic catchall page (posts table)
│   ├── admin/                      # Protected admin dashboard
│   │   ├── forgot-password/        # Password recovery request
│   │   ├── reset-password/         # Password reset form
│   │   ├── layout.tsx              # Admin layout (sidebar)
│   │   ├── page.tsx                # Admin home (dashboard)
│   │   ├── banner/                 # Manage site banners
│   │   ├── popup/                  # Manage pop-ups
│   │   ├── redirects/              # Manage URL redirects
│   │   ├── cache/                  # Cache invalidation tools
```

```
│   │   ├── posts/                 # Standalone content pages
│   │   ├── training/              # Training/courses management
│   │   ├── alpha/                 # Manage Alpha course events
│   │   ├── bible-course/          # Manage The Bible Course events
│   │   ├── cap-money/             # Manage CAP Money Course events
│   │   ├── recovery/              # Manage Recovery course events
│   │   ├── featured-course/       # Choose the What's On featured course
│   │   ├── store/                 # Shop admin (products, orders, hero)
│   │   └── hr/                    # HR staff features (unlinked, in progress)
│   ├── jobs/                      # Job listing & application
│   │   ├── page.tsx               # Job list
│   │   ├── [slug]/page.tsx        # Job detail
│   │   ├── ApplyForm.tsx          # Job application form
│   │   └── actions.ts             # Server actions for applying
│   ├── api/                       # API routes (serverless functions) — no `public/` subfolder
│   │   ├── admin/                 # Admin API endpoints (gated by middleware.ts)
│   │   │   ├── logout/            # End admin session
│   │   │   ├── redirects/         # CRUD redirects
│   │   │   ├── popup/             # CRUD pop-ups
│   │   │   ├── revalidate/        # ISR cache invalidation
│   │   │   ├── posts/, training/, alpha-events/, featured-course/, hr/, store/, shop-hero/
│   │   │   └── ...
│   │   ├── chat/                  # POST /api/chat — Smart Search tool-calling chat
│   │   ├── youtube/                # videos/, thumbnail/[id]/, status/, live/
│   │   ├── alpha-ask/, alpha-events/ # Public Alpha info endpoints
│   │   ├── store/                 # checkout/, checkout/bypass/ (public storefront)
│   │   ├── training/               # unlock/, posts/[id]/timer/
│   │   ├── health/                 # smart-search/ health check
│   │   ├── turnstile/               # verify/ — Cloudflare Turnstile token check (sets ts_verifi
│   │   └── webhooks/               # stripe/ only — no GitHub/Vercel webhook route
│   ├── [slug]/                    # Dynamic catchall (posts table)
│
├── components/                    # React components (shared across pages)
│   ├── ChurchHeader.tsx           # Site header with nav
│   ├── ChurchFooter.tsx           # Site footer
│   ├── FooterLinkGroup.tsx        # Footer link column (accordion on mobile)
│   ├── Providers.tsx              # Client context providers
│   ├── CookieBanner.tsx           # GDPR cookie consent
│   ├── AnalyticsGate.tsx          # Conditional analytics loading
│   ├── SiteBanner.tsx             # Announcement banner (from DB)
│   ├── SitePopup.tsx              # Modal pop-up (from DB)
│   ├── FloatingSmartSearch.tsx    # The one floating widget: AI chat + scripted "New here?"
│   ├── smartSearch/               # Smart Search result cards (products, weather, maps, web)
│   ├── LiveBanner.tsx             # "WE ARE LIVE" banner bar
│   ├── GlassBloomTracker.tsx      # Glass-effect performance tracking
│   ├── FooterGate.tsx / PerformanceGate.tsx / BannerSpacer.tsx  # Layout/perf gating helpers
│   ├── admin/                     # Admin-specific components
│   │   ├── AdminSidebar.tsx       # Admin nav menu
│   │   ├── AdminHeader.tsx        # Admin shell header (breadcrumb + "View live site")
│   │   ├── RichTextEditor.tsx     # Shared TipTap editor — posts, training posts, HR jobs, shop
│   │   ├── training/              # Course management UI (uses RichTextEditor)
│   │   ├── hr/                    # HR management UI (unlinked, in progress)
│   │   ├── posts/                 # Content editor (uses RichTextEditor)
│   │   └── ...
│   ├── shop/                      # Storefront components (ProductCard, ShopProductGrid, ShopHer
│   ├── connect-card/              # Prayer form components
│   ├── kids/                      # Kids ministry components
│   ├── home/                      # Home page components
│   ├── visit/                     # Visit page components
│   ├── serve/                     # Volunteer components
```

```
│   ├── new-here/                 # Onboarding components
│   ├── alpha/                    # Alpha course components
│   ├── AnimateIn.tsx             # Scroll-triggered animations
│   ├── ChurchSuiteEmbed.tsx      # ChurchSuite integration (events)
│   ├── ui/EmbedLoadingOverlay.tsx # Spinner + escalating copy over a loading iframe
│   └── ...
│
├── lib/                          # Utility functions and helpers
│   ├── supabase.ts               # Supabase admin client (server-only)
│   ├── supabase-browser.ts       # Supabase client (browser)
│   ├── podcast.ts                # Buzzsprout RSS feed (sermon audio)
│   ├── sermonPairing.ts          # Matches the latest video to its podcast episode
│   ├── youtube.ts                # YouTube API client
│   ├── smartSearch.ts            # AI search logic (parseAnswer, fallbacks)
│   ├── smartSearch/tools.ts      # Smart Search tool-calling tools (products, weather, maps, we
│   ├── embedLoading.ts           # Stage timings for the embed loading overlay
│   ├── pageContent.ts            # Dynamic page editing
│   ├── posts.ts                  # Dynamic posts/pages
│   ├── training.ts               # Training courses
│   ├── jobs.ts / jobs.server.ts  # Job listing & applications
│   ├── hr.ts                     # HR staff operations
│   ├── shop.ts / shop.server.ts  # Shop types, price helpers, published-product fetchers
│   ├── stripe.ts                 # Stripe client singleton
│   ├── cart-store.ts             # Zustand basket store (localStorage-persisted)
│   ├── checkout.server.ts        # Shared order/pricing logic (checkout, webhook, test bypass)
│   ├── courses.ts                # Alpha/Recovery/Bible Course/CAP course definitions
│   ├── courseEvents.ts           # alpha_events type registry — banner surfaces + admin pages
│   ├── cn.ts                     # className joiner (no Tailwind conflict resolution)
│   ├── welcomeChat.ts            # Scripted "New here?" conversation tree (pure, unit-tested)
│   ├── openaiClient.ts           # OpenAI client + SMART_SEARCH_MODEL constant
│   ├── siteKnowledge.ts          # AI search knowledge base
│   ├── serviceStatus.ts          # Smart Search health / kill-switch state (service_status tabl
│   ├── smartSearchAlertEmail.ts  # Resend email on Smart Search down/recovered transitions
│   ├── rateLimit.ts              # Rate limiting
│   ├── loginRateLimit.ts         # Login attempt limiting
│   ├── turnstile.ts              # Cloudflare Turnstile verification + signed ts_verified cooki
│   ├── podcast.ts                # Podcast metadata
│   ├── accessRequestEmail.ts     # Email templates
│   ├── passwordResetEmail.ts     # Email templates
│   ├── staffLogins.ts            # Staff login records
│   ├── collections.ts            # Content collections
│   ├── sermonPlayerContext.tsx   # Sermon player state
│   ├── sermonSearchContext.tsx   # Sermon search state
│   ├── cookieConsent.tsx         # Cookie preferences
│   ├── alphaSession.ts           # Alpha course sessions
│   ├── trainingAccess.ts         # Training permissions
│   ├── ai/                       # Only media-types.ts remains — the AI page-generation
│   │   └── media-types.ts        # feature (llm-client, code-generator, code-validator,
│   │                              # git-automation, page-audit-email) was removed in 7aa899d
│   │                              # alongside the old "Destiny AI" page. OpenAI is now used
│   │                              # only for Smart Search (lib/openaiClient.ts, lib/smartSearch
│   ├── reserved-slugs.ts         # Protected URL paths
│   └── ...
│
├── supabase/                     # Supabase configuration
│   └── migrations/               # Database schema migrations (34 files) — selected highlights
│       ├── 001_redirects.sql     # URL redirect table
│       ├── 002_hidden_videos.sql # Hidden sermon videos (feature since removed)
│       ├── 003_content.sql       # Site banner & page content
│       ├── 004_banner_type.sql   # Banner types (sitewide, alpha, etc.)
```

```
│       ├── 006_hire_enquiries.sql # Venue hire form submissions
│       ├── 007_alpha_events.sql   # Alpha course events
│       ├── 008_alpha_events_online.sql # Online/hybrid Alpha support
│       ├── 009_alpha_events_frequency.sql # Event recurrence
│       ├── 010_alpha_events_recovery_type.sql # Recovery program
│       ├── 20260329175837_add_subject_to_contact_messages.sql # Contact form subject field
│       ├── 20260502_create_site_popup.sql # Modal pop-ups
│       ├── 20260507_create_builder_media_bucket.sql # AI page builder media (builder since remov
│       ├── 20260514_studio_v2_schema.sql # Page builder schema (removed by 20260711_06_remove_pa
│       ├── 20260531_hr.sql        # HR staff, leave, reviews, documents
│       ├── 20260606_jobs.sql      # Job listings & applications
│       ├── 20260608_service_status.sql # Feature flags
│       ├── 20260614_staff_logins.sql # Staff login audit trail
│       ├── 20260616_training.sql, 20260616_0{2,3,4}_*.sql # Training categories/subgroups/folder
│       ├── 20260618_posts.sql     # Standalone posts (`/[slug]` catch-all)
│       ├── 20260708_shop.sql      # Shop: products, variants, orders, items
│       ├── 20260708_shop_product_fit.sql # products.fit (male/female/unisex/kids)
│       ├── 20260710_shop_hero.sql # Editable auto-rotating /shop hero slides
│       ├── 20260711_02..07_*.sql  # RLS/security-advisor fixes; shop product_type; page builder
│       ├── 20260711_rls_harden_base_tables.sql # RLS hardening pass across base tables
│       ├── 20260712_alpha_events_bible_course_type.sql # The Bible Course
│       ├── 20260712_02_featured_course.sql # Featured course (What's On)
│       ├── 20260728_featured_event.sql   # Featured ChurchSuite event + its popup
│       └── 20260807_alpha_events_cap_type.sql # CAP Money Course
│
├── utils/                         # Utility modules
│   ├── supabase/                  # Supabase client factories
│   │   ├── service.ts             # Service role client
│   │   └── ...
│   └── ...
│
├── contexts/                      # React context definitions
├── docs/                          # Additional documentation, incl. mobile-app-scope.md (+ .pdf
│   └── content/                   # Source copy for policy/partner text — the live pages render
│                                   # an edited subset, so these are the fuller source. See its R
├── mobile/                        # Native SwiftUI iOS app (Swift 6, strict concurrency). Five t
│                                   # Home/Sermons/Events/Give/More. XcodeGen project (project.ym
│                                   # the source of truth; the .xcodeproj is generated, not commi
│                                   # Talks only to the /api/app/v1/* BFF — no @destiny/shared im
│                                   # the BFF does all normalising. Build via `make` (see mobile/
│                                   # Asset catalog is generated from brand tokens: `npm run gen:
├── packages/                      # npm workspaces (root package.json `workspaces: ["packages/*"
│   └── shared/                    # @destiny/shared — framework-agnostic types/logic shared by t
│                                   # app and the app BFF (the Swift app can't import TS). Ships
│                                   # (Next transpiles it via `transpilePackages`). Modules under
│                                   # churchsuite/{events,dates,series,sanitize,ics} and design/t
│                                   # (canonical DC brand palette/typography matching app/globals
├── types/
│   └── turnstile.d.ts             # Declares window.turnstile (Cloudflare Turnstile JS API)
├── tests/                         # Playwright E2E specs (contact, cookies, give,
│                                   # navigation, sermons) — run via `npx playwright test`
├── public/                        # Static files
│   ├── img/                       # Church logos, backgrounds
│   ├── og/                        # Open Graph images (social share)
│   ├── fonts/                     # Custom fonts
│   └── ...
│
├── .github/                       # GitHub workflows
│   └── workflows/                 # CI/CD pipelines
│
```

```
├── .claude/                    # Claude Code settings
├── .vscode/                    # VS Code workspace settings
├── next.config.ts              # Next.js configuration
├── playwright.config.ts        # E2E test configuration
├── eslint.config.mjs           # ESLint rules
├── postcss.config.mjs          # PostCSS plugins
├── tsconfig.json               # TypeScript compiler options
├── package.json                # Node dependencies and scripts
├── package-lock.json           # Locked versions
├── .gitignore                  # Git exclusions
├── CLAUDE.md                   # Project instructions
├── README.md                   # Public project README
└── REPOSITORY_DOCUMENTATION.md # This file
```

Database Schema

Overview

The database uses Supabase (managed PostgreSQL) with Row-Level Security (RLS) for sensitive data. Public tables are readable via the anonymous key; member-only and admin tables use the service role key (server-to-database communication).

Tables

1. redirects

Purpose: URL redirect management (e.g., `/prayer-request` → `/connect-card`)

```
CREATE TABLE redirects (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug text UNIQUE NOT NULL,            -- /short-path
  target_url text NOT NULL,             -- https://example.com or /absolute/path
  label text,                           -- Display name (unused currently)
  active boolean DEFAULT true,          -- Toggle visibility
  created_at timestamptz DEFAULT now()
);

-- RLS: Public read access to active redirects; service role has full access
```

Used By: - `app/api/[slug]` or Next.js `redirects()` config during build - Admin dashboard to manage redirects

---

3. site_banner

Purpose: Sitewide announcement banners (top of every page)

```sql
CREATE TABLE site_banner (
  id uuid PRIMARY KEY,
  message text NOT NULL,              -- Banner text (HTML allowed)
  active boolean DEFAULT false,       -- Toggle on/off
  type text,                          -- 'sitewide', 'alpha', 'youth_alpha', 'recovery', 'bible_
  link text,                          -- Optional URL for banner
  link_text text,                     -- CTA button text
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz
);

-- Types reference alpha_events for dynamic data (e.g., next Alpha start date)
-- RLS: Service role only
```

Used By: - `app/layout.tsx` fetches active banner on every request - Admin dashboard to edit banners

---

4. page_content

Purpose: Key-value store for editable site content

```sql
CREATE TABLE page_content (
  page text NOT NULL,                 -- 'give', 'general', etc.
  key text NOT NULL,                  -- 'account_name', 'service_time'
  value text DEFAULT '',              -- Content value
  updated_at timestamptz DEFAULT now(),
  PRIMARY KEY (page, key)
);

-- Example rows:
-- ('give', 'account_name', 'Destiny Church Tees Valley')
-- ('give', 'sort_code', '08-92-99')
-- ('general', 'service_time', '11am')
-- RLS: Service role only
```

Used By: - `lib/pageContent.ts` to fetch dynamic content - Admin dashboard to edit giving details, service times, etc.

---

5. contact_messages

Purpose: Messages from the public contact form

```sql
CREATE TABLE contact_messages (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  email text NOT NULL,
  subject text,                       -- Added later (migration 20260329)
  message text NOT NULL,
  created_at timestamptz DEFAULT now()
);

-- RLS: Public insert (form submission); service role read
```

Used By: - `app/contact/actions.ts` (server action) inserts submissions - Admin dashboard to view inquiries

---

6. hire_enquiries

Purpose: Venue hire request submissions

```
CREATE TABLE hire_enquiries (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  email text NOT NULL,
  phone text NOT NULL,
  date_needed date NOT NULL,
  event_description text NOT NULL,
  approx_guests integer,
  created_at timestamptz DEFAULT now()
);

-- RLS: Public insert; service role read
```

Used By: - `app/hire/actions.ts` (server action) inserts submissions - Admin dashboard to view inquiries

---

7. alpha_events

Purpose: Alpha course event scheduling and details

```
CREATE TABLE alpha_events (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  type text NOT NULL,                  -- 'alpha', 'youth_alpha', 'recovery', 'bible_course'
  active boolean DEFAULT true,
  start_date date NOT NULL,
  signup_url text,                     -- External booking link
  format text,                         -- 'in_person', 'online', 'hybrid'
  location text,                       -- 'Destiny Centre', etc.
  meeting_platform text,               -- 'Zoom', 'Teams', etc. (if online)
  frequency text DEFAULT 'weekly',     -- 'weekly', 'fortnightly'
  custom_interval_days integer,        -- Override frequency
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz
);

-- RLS: Public read; service role write
```

Used By: - `app/layout.tsx` fetches to populate site-wide banner - `/alpha`, `/destiny-recovery`, `/bible-course` and `/cap-money` pages display their next upcoming event - Admin (`/admin/alpha`, `/admin/recovery`, `/admin/bible-course`, `/admin/cap-money`) to manage event dates and URLs

> The `bible_course` and `cap` types are shared infrastructure for The Bible Course (Bible Society) and the CAP Money Course (Christians Against Poverty) — they reuse this table and the `/api/admin/alpha-events` routes rather than adding new ones. Added by migrations `20260712_alpha_events_bible_course_type.sql` and `20260807_alpha_events_cap_type.sql`.

> The type list lives in **`lib/courseEvents.ts`** (`COURSE_EVENT_TYPES`) and drives every banner surface *and* all four admin pages. Adding a type there without the matching `alpha_events_type_check` migration means inserts fail with a check-constraint error.
>
> **Adding a course is now three steps:** the migration, a `COURSE_EVENT_META` entry, and a `COURSE_ADMIN_PAGES` entry plus a four-line route file. See *Course admin pages* under Components.

7b. featured_course

Purpose: Singleton setting for which course headlines the What's On "Courses" section.

```
CREATE TABLE featured_course (
  id integer PRIMARY KEY DEFAULT 1,        -- always 1 (singleton)
  course_id text NOT NULL DEFAULT 'bible_course'
    CHECK (course_id IN ('bible_course','cap','alpha','recovery')),
  updated_at timestamptz DEFAULT now()
);
-- RLS: enabled, deny-all ("service only"); server routes use the service key.
```

The four courses are defined in `lib/courses.ts` (each with a grid-card and a full-width "featured" config). The featured course renders as the wide banner; the other three render as cards, so all four are always visible. Chosen at `/admin/featured-course`.

Used By: - `app/whats-on/page.tsx` reads it and passes `featuredId` to `CoursesSection` - `app/api/admin/featured-course` (GET/PUT) — PUT revalidates `/whats-on` - Migration `20260712_02_featured_course.sql`

7b. featured_event

Purpose: The one ChurchSuite event promoted across the site — banner on What's On, first card on the homepage, and a site-wide popup.

Events are **not stored in this database** (they come live from the ChurchSuite calendar feed), so this holds a *pointer* to a feed event plus the copy that overrides it. `lib/events.server.ts` merges the overrides over live feed data at render time; anything left blank falls back to ChurchSuite.

```sql
CREATE TABLE featured_event (
  id integer PRIMARY KEY DEFAULT 1,          -- singleton, CHECK (id = 1)
  -- target
  event_identifier text,                     -- ChurchSuite occurrence id — the lookup key
  event_sequence   integer,                  -- series key (null for one-offs)
  event_id         integer,
  event_name       text,                     -- snapshots, for admin display and
  event_slug       text,                     --   the feed-outage fallback
  event_ends_at    timestamptz,
  active        boolean NOT NULL DEFAULT false,
  promote_from  timestamptz,                 -- optional scheduling window
  promote_until timestamptz,
  -- hero overrides (all nullable → fall back to the feed)
  headline text, blurb text, image_url text, image_path text,
  cta_text text, cta_link text,
  -- event popup
  popup_active boolean NOT NULL DEFAULT false,
  popup_title text, popup_body text,
  popup_image_url text, popup_image_path text,
  popup_cta_text text, popup_cta_link text,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
-- RLS: enabled, deny-all ("service only"); server routes use the service key.
```

Why the hero and popup share one row rather than living in two tables: the popup must always advertise the same event as the hero. Two tables means either a duplicated `event_identifier` that drifts, or a foreign key that reduces to one row anyway — and auto-expiry plus the promote window have to apply identically to both surfaces. Two admin pages write to the one row, which is why `/api/admin/featured-event/popup` issues a **partial** update of `popup_*` only: a full-row upsert from that page would blank every hero override.

Constraints worth knowing: `active` requires an `event_identifier`; `popup_active` requires `active` (the event-popup admin page catches this and explains it rather than surfacing a raw Postgres error); `promote_until > promote_from`.

Expiry is computed in code, not SQL. The feed is forward-only, so a finished event simply stops appearing and the promotion ends. `event_ends_at` exists for the outage case: `fetchChurchSuiteEvents` returns `[]` on *any* error, which would otherwise silently un-feature the event for five minutes — so when the feed is empty but the stored copy is complete and the event has not ended, the hero renders from the snapshot.

Images reuse the existing public `popup-images` bucket with a `featured-event-` / `event-popup-` filename prefix, so there is no new bucket or storage policy.

Used By: - `lib/events.server.ts` — `getFeaturedEvent()`, `getActiveEventPopup()` - `components/events/FeaturedEventHero.tsx` (What's On), `components/home/WhatsOnSection.tsx` (carousel pin) - `components/events/EventPopup.tsx`, wired in `app/layout.tsx` - `app/api/admin/featured-event` (GET/PUT, revalidates `/whats-on` and `/`) and `.../popup` (PUT) - Migration `20260728_featured_event.sql`

---

8. site_popup

Purpose: Modal pop-ups that appear once per session/visitor

```
CREATE TABLE site_popup (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  active boolean DEFAULT false,
  title text,
  body text,                           -- Supports markdown/HTML
  cta_text text,                       -- Button label
  cta_link text,                       -- Button URL
  image_url text,                      -- Popup image
  show_once boolean DEFAULT true,      -- Show only once per browser session
  updated_at timestamptz DEFAULT now()
);

-- RLS: Service role only
```

Used By: - `app/layout.tsx` fetches active pop-up - Displayed by `SitePopup.tsx` component - Admin dashboard to manage pop-ups

---

9. nfc_tiles

Purpose: The admin-managed tiles on `/nfc`, the "digital back of seats" page

```
CREATE TABLE nfc_tiles (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  title text NOT NULL,                 -- ≤ 60 chars (DB check)
  subtitle text,                       -- ≤ 90 chars, the line under the title on the card
  icon text NOT NULL DEFAULT 'star',   -- Material Symbols Rounded ligature
  mode text NOT NULL DEFAULT 'info'    -- 'embed' = ChurchSuite form, 'info' = copy + CTA,
    CHECK (mode IN ('embed','info','event')),  -- 'event' = a ChurchSuite event's signup
  embed_url text,                      -- Required when mode = 'embed'; the signup URL when 'ever
  embed_size text NOT NULL DEFAULT 'md' CHECK (embed_size IN ('md','lg')),
  body text,                           -- ≤ 600 chars, mode = 'info'
  image_url text, image_path text,     -- Shared `popup-images` bucket, `nfc-` prefix
  cta_text text, cta_link text,        -- Link through to a full page on the site
  -- mode = 'event': a pointer into the ChurchSuite feed plus snapshots
  event_identifier text,               -- Occurrence identifier — the feed lookup key
  event_sequence integer,              -- Series key (null for one-offs); durable fallback lookuj
  event_slug text,                     -- Resolved /whats-on slug
  event_name text,                     -- For the admin list without hitting the feed
  event_ends_at timestamptz,           -- End of the last occurrence — drives auto-expiry
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- CHECK (mode <> 'event' OR (event_identifier IS NOT NULL AND embed_url IS NOT NULL))
-- RLS: public read of active rows; writes service-role only
```

Why the two fixtures aren't rows: Connect Card and Giving are hardcoded as `PINNED_TILES` in `lib/nfcTiles.ts` and always render first. They have to survive an empty table, an unrun migration, or a Supabase blip ten minutes into a service — and a row with a delete button next to it is a row that eventually gets deleted. `/admin/nfc` shows them as read-only "Always shown" entries so it's obvious why they can't be edited there.

Why event tiles reuse `embed_url`: an event tile *is* an embed tile once resolved — the popup frames the event's ChurchSuite signup form the same way the Connect Card tile frames its form. So the resolved signup URL lands in `embed_url` rather than a column of its own: the renderer needs one widened condition instead of a second code path, and the row stays renderable when the feed is unreachable. `event_ends_at` is what hides the tile once the event has run; the row itself stays, so `/admin/nfc` can show an "Ended" pill and offer a repoint instead of leaving a mystery gap.

Used By: - `lib/nfcTiles.server.ts` → `getNfcTiles()`, read by `app/nfc/page.tsx` - `app/api/admin/nfc/route.ts` + `app/admin/nfc/page.tsx` for CRUD - `app/api/admin/events/route.ts` supplies the event picker (shared with `/admin/featured-event`)

---

10. hr_staff

Purpose: Employee/volunteer directory

```
CREATE TABLE hr_staff (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name text NOT NULL,
  last_name text NOT NULL,
  email text,
  phone text,
  job_title text,
  department text,
  employment_type text NOT NULL DEFAULT 'full_time'
    CHECK (employment_type IN ('full_time','part_time','volunteer','contractor')),
  status text NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','on_leave','left')),
  start_date date,
  end_date date,
  annual_leave_entitlement numeric DEFAULT 0,
  notes text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Trigger: hr_staff_updated_at auto-updates `updated_at` on every change
-- RLS: Service role only (accessed via proxy requiring auth)
```

Used By: - HR admin to manage staff directory - `/admin/hr` dashboard

---

10b. admin_roles

Purpose: Access levels for `/admin` — five independent booleans per admin login, checked by `middleware.ts` on every `/admin/*` and `/api/admin/*` request

```
CREATE TABLE admin_roles (
  auth_user_id uuid PRIMARY KEY REFERENCES auth.users (id) ON DELETE CASCADE,
  email text,
  training_admin boolean NOT NULL DEFAULT false,
  event_admin boolean NOT NULL DEFAULT false,
  store_admin boolean NOT NULL DEFAULT false,
  site_admin boolean NOT NULL DEFAULT false,
  super_admin boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

-- RLS: Service role only (deny-all "service only" policy, same as every other table)
```

Distinct from the legacy, unused `admin_users` table in `supabase/schema.sql` (username/password_hash — predates the Supabase Auth migration; not read by any app code). Managed by Super Admins at `/admin/users`. See Authorization Layers for the role → route mapping and `lib/adminRoles.ts` for the enforcement logic.

Used By: - `middleware.ts` — role check on every admin request - `/admin/users` + `app/api/admin/users/**` — role management UI - `app/api/admin/me/roles` — lets the sidebar know what to show

---

11. hr_leave_requests

Purpose: Time-off and leave requests

```
CREATE TABLE hr_leave_requests (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id uuid NOT NULL REFERENCES hr_staff(id) ON DELETE CASCADE,
  type text NOT NULL DEFAULT 'holiday'
    CHECK (type IN ('holiday','sick','other')),
  start_date date NOT NULL,
  end_date date NOT NULL,
  days numeric DEFAULT 0,              -- Calculated number of working days
  reason text,
  status text NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending','approved','rejected')),
  reviewed_by text,                    -- Name of approver
  reviewed_at timestamptz,
  created_at timestamptz DEFAULT now()
);

-- RLS: Service role only
-- Indexes: staff_id, status
```

Used By: - HR staff to request/approve leave - Admin dashboard for leave tracking

---

12. hr_documents

Purpose: HR document library (contracts, policies, payslips)

```
CREATE TABLE hr_documents (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id uuid REFERENCES hr_staff(id) ON DELETE CASCADE,  -- NULL = org-wide
  title text NOT NULL,
  category text NOT NULL DEFAULT 'other'
    CHECK (category IN ('contract','policy','payslip','other')),
  file_path text NOT NULL,             -- Supabase Storage path
  file_name text NOT NULL,
  mime_type text,
  size_bytes bigint,
  uploaded_by text,
  created_at timestamptz DEFAULT now()
);

-- RLS: Service role only
-- Indexes: staff_id
-- Files stored in 'hr-documents' private bucket (signed URLs only)
```

Used By: - Admin to upload and organize HR documents - Staff to download their documents via signed URLs

---

13. hr_reviews

Purpose: Appraisals and 1-to-1 meeting records

```
CREATE TABLE hr_reviews (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id uuid NOT NULL REFERENCES hr_staff(id) ON DELETE CASCADE,
  review_date date NOT NULL,
  type text NOT NULL DEFAULT 'one_to_one'
    CHECK (type IN ('appraisal','one_to_one')),
  reviewer text,                       -- Name of reviewer
  summary text,                        -- Notes/outcomes
  next_review_date date,               -- Scheduled follow-up
  created_at timestamptz DEFAULT now()
);

-- RLS: Service role only
-- Indexes: staff_id
```

Used By: - Admin to log reviews - HR dashboard to track review schedule

---

14. jobs

Purpose: Job listings and applications

```sql
CREATE TABLE jobs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,                 -- Job title
  slug text UNIQUE NOT NULL,           -- URL slug
  description text NOT NULL,           -- Full job description (markdown)
  department text,                     -- 'Worship', 'Admin', etc.
  employment_type text,                -- 'full_time', 'part_time'
  salary_range text,                   -- Free text (e.g., "£25,000-£30,000")
  closing_date date,                   -- Application deadline
  active boolean DEFAULT true,         -- Hide expired jobs
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- RLS: Public read (active only); service role write
-- Indexes: slug, active
```

Used By: - `/jobs` page to list open positions - `/jobs/[slug]` for job detail pages - Admin to create/edit job postings

---

15. job_applications

Purpose: Submitted job applications

```sql
CREATE TABLE job_applications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id uuid NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  name text NOT NULL,
  email text NOT NULL,
  phone text NOT NULL,
  cv_url text,                         -- Uploaded CV (signed URL)
  cover_letter text,                   -- Freeform text
  status text DEFAULT 'pending'
    CHECK (status IN ('pending','reviewing','rejected','offered')),
  created_at timestamptz DEFAULT now()
);

-- RLS: Public insert (application form); service role read
-- Indexes: job_id, status
```

Used By: - Job application form (`app/jobs/ApplyForm.tsx`) - Admin dashboard to review applications

---

16. service_status

Purpose: Runtime health / kill-switch state for backend services (currently only `smart_search`). The self-healing health check (`GET /api/health/smart-search`) writes this row; the site reads `enabled` to decide whether to render Smart Search.

```
CREATE TABLE service_status (
  service              text PRIMARY KEY,    -- 'smart_search'
  enabled              boolean NOT NULL DEFAULT true,
  reason               text,                -- why it was last disabled (OpenAI error, etc.)
  last_check_at        timestamptz,
  next_check_at        timestamptz,         -- daily when healthy, hourly while down
  consecutive_failures  int NOT NULL DEFAULT 0,
  updated_at           timestamptz NOT NULL DEFAULT now()
);

-- RLS enabled with NO policies: only the service-role client (which bypasses
-- RLS) touches this table; anon/authenticated get no access.
```

Migration: `supabase/migrations/20260608_service_status.sql`.

Used By: - `lib/serviceStatus.ts` — `getSmartSearchStatus()` / `isSmartSearchEnabled()` (reads **fail open**: a status-store blip never disables the feature) and `setSmartSearchStatus()` - `app/layout.tsx` — reads `isSmartSearchEnabled()` to decide whether to mount `FloatingSmartSearch` - `GET /api/health/smart-search` — the cron health check that flips `enabled` and schedules the next check

---

17. posts

Purpose: Generic published pages served by the `/[slug]` catch-all route (freeform site pages outside the main nav)

```
CREATE TABLE posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  slug text UNIQUE NOT NULL,
  body text,
  is_published boolean NOT NULL DEFAULT false,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- RLS: Service role only; public read happens server-side via lib/posts.server.ts
```

Used By: - `lib/posts.server.ts` (`getPublishedPostBySlug`) for the public `/[slug]` catch-all - `app/api/admin/posts` CRUD, admin dashboard to write pages

---

17b. training_categories / training_subgroups / training_folders / training_posts

Purpose: The `/training` member resource library — a category → sub-group (optionally password-protected) → folder → post hierarchy.

```sql
CREATE TABLE training_categories (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL, slug text UNIQUE NOT NULL, description text, icon text,
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT true,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_subgroups (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  category_id uuid NOT NULL REFERENCES training_categories(id),
  name text NOT NULL, slug text NOT NULL, description text,
  password_hash text,                   -- NULL = no password gate; never returned to clients, or
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT true,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_folders (
  id uuid PRIMARY KEY DEFAULT extensions.uuid_generate_v4(),
  subgroup_id uuid NOT NULL REFERENCES training_subgroups(id),
  name text NOT NULL, sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  subgroup_id uuid NOT NULL REFERENCES training_subgroups(id),
  folder_id uuid REFERENCES training_folders(id),   -- NULL = ungrouped within the subgroup
  title text NOT NULL, slug text NOT NULL, summary text, body text,
  min_read_seconds integer NOT NULL DEFAULT 0,
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT false,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

-- RLS: Service role only. Public read of published rows happens server-side
-- via lib/training.server.ts; password_hash is never selected into client-facing shapes.
```

Used By: - `lib/training.server.ts` (`getTrainingTree`) for the public `/training` tree - `app/api/training/unlock` to verify a sub-group password - `app/api/training/posts/[id]/timer` to enforce `min_read_seconds` before a post can be marked complete (HMAC-signed read timer; see Training API) - `app/api/admin/training/{categories,subgroups,folders,posts}` CRUD, admin dashboard

---

Row-Level Security (RLS) Strategy

18. products / product_variants / orders / order_items (Shop)

Purpose: The Stripe-powered store (`/shop`), replacing the old WooCommerce site. Physical apparel with size + colour variants and per-variant stock; collection-only fulfilment. Prices are integer **pennies** (GBP). Migration: `supabase/migrations/20260708_shop.sql` , extended by `20260708_shop_product_fit.sql` (`fit`) and `20260711_05_shop_product_type.sql` (`product_type`).

```
CREATE TABLE products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug text UNIQUE NOT NULL,
  name text NOT NULL,
  description text,
  base_price_pennies integer NOT NULL DEFAULT 0,
  category text,
  fit text NOT NULL DEFAULT 'unisex'          -- 'male'|'female'|'unisex'|'kids' — drives size ch
    CHECK (fit IN ('male','female','unisex','kids')),
  product_type text NOT NULL DEFAULT 'clothing'  -- 'clothing'|'books'|'other' — colour/size mat
    CHECK (product_type IN ('clothing','books','other')),
  images jsonb NOT NULL DEFAULT '[]',   -- [{ url, path, alt }]
  is_published boolean NOT NULL DEFAULT false,
  is_featured boolean NOT NULL DEFAULT false,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

CREATE TABLE product_variants (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id uuid REFERENCES products(id) ON DELETE CASCADE,
  size text, color text, color_hex text, sku text,
  price_pennies integer,                -- null → inherit product base price
  stock integer NOT NULL DEFAULT 0,
  is_active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  UNIQUE (product_id, size, color)
);

CREATE TABLE orders (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_number text UNIQUE NOT NULL,    -- human-friendly, e.g. DT-7F3K9Q2X
  stripe_payment_intent_id text UNIQUE,
  customer_name text NOT NULL, customer_email text NOT NULL, customer_phone text,
  notes text,
  status text NOT NULL DEFAULT 'pending',  -- pending|paid|fulfilled|cancelled|refunded
  fulfillment_method text NOT NULL DEFAULT 'collection',
  subtotal_pennies integer, total_pennies integer, currency text DEFAULT 'gbp',
  paid_at timestamptz, created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE order_items (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id uuid REFERENCES orders(id) ON DELETE CASCADE,
  product_id uuid, variant_id uuid,     -- snapshot fields survive product deletion
  product_name text NOT NULL, size text, color text,
  unit_price_pennies integer NOT NULL, quantity integer NOT NULL DEFAULT 1
);
-- RLS: single "service only" policy on all four (like jobs). Public storefront
-- reads via lib/shop.server.ts; checkout/webhook write via the service role.
-- Storage: public `product-images` bucket for product photos (WebP).
```

Used By: `/shop` storefront, `/admin/store` CRUD, `POST /api/store/checkout`, `POST /api/webhooks/stripe`. `description` is HTML written via the shared `RichTextEditor` (same component used for posts/training) — the admin editor no longer has a markdown Write/Preview toggle, and the public product page renders it with `dangerouslySetInnerHTML` (no markdown fallback).

---

19. shop_hero_slides (Shop hero)

Purpose: Dynamic, admin-editable hero at the top of `/shop`. Multiple active slides auto-rotate (crossfade) on the storefront; with no active slides the shop shows its static "The Destiny Store" masthead. Migration: `supabase/migrations/20260710_shop_hero.sql`.

```
CREATE TABLE shop_hero_slides (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  active boolean NOT NULL DEFAULT true,
  heading text, subheading text,
  cta_text text, cta_link text,
  image_url text, image_path text,      -- storage path in shop-hero-images bucket
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);
-- RLS: single "service only" policy (like the shop tables). Public storefront
-- reads active slides via lib/shop.server.ts → getActiveShopHeroSlides().
-- Storage: public `shop-hero-images` bucket for hero backgrounds (WebP).
```

Used By: `/shop` storefront (`components/shop/ShopHero.tsx`), `/admin/store/hero` CRUD.

---

20. Base-schema & legacy tables (`supabase/schema.sql`)

`supabase/schema.sql` defines the original **base-schema** tables that predate the migration-per-feature workflow. They still exist in the live database (and are recreated by a fresh rebuild), but are **not read by the current Next.js app** — the live site serves sermons directly from the YouTube Data API (`lib/youtube.ts`), so these DB tables are effectively legacy/service-only and are populated (if at all) by external tooling rather than the web app:

Table	Shape (key columns)	Status
`sermons`	`id`, `title`, `date`, `podcast_*`, `youtube_video_id`, `summary`, `summary_points` (jsonb), `transcript`	Legacy sermon catalogue — app now reads from YouTube API, not this table
`sermon_transcripts`	`sermon_id` (→ sermons), `segments` (jsonb), `status`	Legacy per-sermon transcript segments
`sermon_link_suggestions`	`podcast_guid`, `youtube_video_id`, `status`	Legacy podcast↔YouTube link matching
`ai_reports`	`sermon_id`, `issue_type`, `name`, `email`, `description`	Legacy "report an issue" submissions
`auth_users`	`email` (unique), `last_login`	Legacy auth bookkeeping (current auth is Supabase Auth)
`admin_users`	`username`, `password_hash`	Legacy admin credentials (current admin auth is Supabase Auth)

All base-schema tables have RLS enabled with a deny-all `"service only"` policy (see the `do $$ … $$` block at the foot of `schema.sql`, codified for rebuilds by `supabase/migrations/20260711_rls_harden_base_tables.sql`). The service/secret key bypasses RLS; the public anon key gets nothing.

Orphaned Studio tables: `studio_assets` and `studio_components` were created by `supabase/migrations/20260514_studio_v2_schema.sql` for an experimental page/Studio builder. The builder was later removed, but `20260711_06_remove_page_builder.sql` only drops `builder_pages` / `builder_templates` / `builder_media` — the two `studio_*` tables were **never dropped**, so they still exist in the database (RLS service-only) as dead schema. They are unused by the app.

All tables have RLS enabled. Access rules:

Table	Public Read	Authenticated Read	Service Role	Purpose
redirects	Yes	-	Yes	Public navigation
site_banner	Yes	-	Yes	Show on every page
page_content	-	-	Yes	Protect sensitive values
contact_messages	-	-	Yes	Protect submissions
hire_enquiries	-	-	Yes	Protect submissions
alpha_events	Yes	-	Yes	Public event dates
featured_course	-	-	Yes	Featured course setting (read via server component)
site_popup	-	-	Yes	Protect pop-up content
nfc_tiles	Yes	-	Yes	Public tiles on /nfc (read via server component)
hr_* (staff, leave, reviews, docs)	-	-	Yes	Sensitive HR data
jobs	Yes	-	Yes	Public listings
job_applications	-	-	Yes	Protect applications
service_status	-	-	Yes	Service health / Smart Search kill-switch (service-role only)
posts	-	-	Yes	Freeform pages (public read via server components)
training_categories / training_subgroups / training_folders / training_posts	-	-	Yes	/training resource library (public read via server components; sub-group passwords hashed)
products / product_variants	-	-	Yes	Shop catalogue (public read via server components)
orders / order_items	-	-	Yes	Store orders (written by Stripe webhook)
shop_hero_slides	-	-	Yes	Editable /shop hero (public read via server components)
sermons / sermon_transcripts / sermon_link_suggestions / ai_reports / auth_users / admin_users	-	-	Yes	Base-schema legacy tables (deny-all "service only"; not read by the app)
studio_assets / studio_components	-	-	Yes	Orphaned Studio-builder tables (never dropped; unused)

Key Point: No table uses authenticated user RLS. All member-facing features use API proxy routes that enforce authentication in application code, then access the database with the service role key. This gives finer control and better error messages.

Core Application Layers

Layer 1: Root Layout (`app/layout.tsx`)

The root layout wraps every page in the application. It:

1. **Defines metadata & SEO** — Title templates, description, Open Graph images for social share
2. **Fetches server data** — Banner, pop-up, feature flags (called on every request)
3. **Renders custom SVG filter** — A sophisticated glass refraction effect used for visual polish
4. **Wraps in providers** — Context providers for auth, theme, analytics
5. **Loads fonts** — Roboto, Anton, Playfair Display from Google Fonts
6. **Sets up structure** — Header, main content area, footer

Key Code:

```
// Fetch active banner (shown at top of site)
async function getActiveBanner() {
  const supabase = createServiceClient();
  const { data: rows } = await supabase
    .from("site_banner")
    .select("active, message, type, link, link_text")
    .eq("active", true);

  // If multiple banners, prioritize sitewide > event banners
  const sitewide = banners.find((b) => b.type === "sitewide");
  if (sitewide) return sitewide;

  // Otherwise return first active event banner (Alpha, Youth Alpha, Recovery)
  // ...
}

// Fetch active pop-up (modal overlay)
async function getActivePopup() {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("site_popup")
    .select("active, title, body, cta_text, cta_link, image_url, show_once")
    .eq("active", true)
    .limit(1)
    .maybeSingle();  // Returns null if no results
  return data ?? null;
}

// Render root HTML (simplified — the real tree also renders LiveBanner,
// GlassBloomTracker, FooterGate/PerformanceGate wrappers, and passes a
// server-seeded `live` status into Providers; see components/ for each)
export default async function RootLayout({ children }) {
  const banner = await getActiveBanner();
  const popup = await getActivePopup();
  const smartSearchEnabled = await isSmartSearchEnabled();

  return (
    <html>
      <body className="...">
        <svg aria-hidden>
          {/* Glass refraction filter (SVG filters for visual effects) */}
        </svg>

        <Providers banner={banner}>
          <CookieBanner />
          <div className="flex flex-col">
            <ChurchHeader />
            <main>{children}</main>
            <ChurchFooter />
          </div>
          <AnalyticsGate />        {/* Conditionally load Vercel Analytics */}
          <SitePopup popup={popup} />
          <FloatingSmartSearch searchEnabled={smartSearchEnabled} />  {/* AI chat + guided script
        </Providers>

        <SpeedInsights />             {/* Vercel performance monitoring */}
      </body>
    </html>
```

```
  );
}
```

SVG Glass Refraction Filter Explanation:

The `<filter id="glass-refract">` is a sophisticated visual effect:

1. **feGaussianBlur** — Blur the alpha channel to create a soft edge ramp
2. **feOffset** — Offset the blurred version left/right and up/down to detect edges
3. **feComposite** — Subtract offsets to create signed X/Y displacement vectors
4. **feDisplacementMap** — Apply the displacement at three scales (42/48/54) to separate color channels slightly
5. **feBlend** — Recombine RGB channels; the offset makes colors separate at edges (chromatic dispersion)

Result: Elements with `.glass-refract` backdrop-filter get a thick glass effect with color separation at edges.

Layer 2: Providers (`components/Providers.tsx`)

Wraps children with React context providers:

- **Supabase Client Provider** — Browser auth token management
- **Theme Provider** — Dark/light mode
- **Query Client** (if using) — API caching
- **Toast Provider** (`components/ToastProvider.tsx`) — Passive notifications via `useToast()` (`success` / `error` / `info`)
- **Dialog Provider** (`components/DialogProvider.tsx`) — Styled, promise-based modal dialogs via `useDialog()`

Dialogs & notifications — no native browser dialogs

Native `alert()` , `confirm()` , and `window.prompt()` are **banned** in this codebase — they look untrustworthy, can't be styled, and freeze the tab. Use the in-app equivalents instead:

- **Passive message** → `useToast()` from `components/ToastProvider.tsx` : `toast.error("…")` , `toast.success("…")` , `toast.info("…")` .
- **Yes/No decision** → `useDialog().confirm(opts)` → `Promise<boolean>` . Renders a centered, blocking styled modal. Options: `{ title?, message, confirmLabel?, cancelLabel?, tone? }` where `tone: "danger"` gives a red confirm button for destructive actions. Pattern: `if (!(await confirm({ message: "Delete this?", tone: "danger", confirmLabel: "Delete" }))) return;` (the enclosing handler must be `async`).
- **Free-text input** → `useDialog().prompt(opts)` → `Promise<string | null>` (`null` = cancelled, matching native `prompt`). Options: `{ title?, message?, placeholder?, defaultValue?, confirmLabel? }` .

Both dialog helpers reuse the visual conventions from `components/admin/hr/HrUI.tsx` (backdrop + rounded card + shared button/input classes). Escape and backdrop-click cancel.

Layer 3: Header & Navigation (`components/ChurchHeader.tsx`)

Rendered on every page (server component with Suspense).

- **Logo** — Clickable link to home
- **Navigation menu** — Links to all main pages
- **Mobile menu** — Hamburger on small screens
- **Search bar** — Filters sermons/content (client-side with fuse.js)
- **Auth indicator** — Login/logout buttons

Layer 4: Main Content (Route-Specific Components)

Each page route (`app/*/page.tsx`) renders page-specific content. Examples:

`/app/sermons/page.tsx` — Sermons

Video for the latest message, audio for everything else. Structure, top to bottom: photo hero with podcast-platform chips → **featured message** → **All episodes** archive → "Every sermon, on the big screen" YouTube band → `WorshipWithUsSection` . It uses the same light bands, container widths and card shell as every other content page — it was hard-coded dark until August 2026, which made it look like a different site, and there is no dark mode here to opt into.

The featured card (`components/sermons/FeaturedSermon.tsx`) opens on **Watch** — a privacy-enhanced `youtube-nocookie` embed behind `VideoConsentGate` — with a **Listen** tab for the same message's podcast audio. Only the active pane is mounted; unmounting the iframe is what stops video playback on switch. Starting audio anywhere on the page flips the card to Listen.

Pairing the two feeds. Video comes from the YouTube Data API (`lib/youtube.ts`), audio from the Buzzsprout RSS feed (`lib/podcast.ts`), and nothing joins them — `supabase/schema.sql` 's `sermons` table would, but the app does not read it. `lib/sermonPairing.ts` matches the latest video to an episode on publish-date proximity plus title-word overlap (boilerplate like "Sunday Service" and speaker suffixes are stripped first). No confident match falls back to the newest episode rather than disabling the toggle, so Listen is never a dead control. Whichever episode is featured is filtered out of the archive so it cannot appear twice.

Archive (`components/sermons/podcast/EpisodeList.tsx`) — audio only, client-side search over title/speaker/summary, 12 per "Load more". `PodcastPlayerProvider` owns the single `<audio>` element, the docked bar and the mobile now-playing sheet. Those stay dark on purpose (a media surface, like Spotify's) but are tinted with the brand `#363f48` rather than an off-palette near-black. The dock publishes its height as `--podcast-dock-height` , which the root layout uses as bottom padding so the dock never covers the footer, and `FloatingSmartSearch` uses to lift itself clear.

`/app/[slug]/page.tsx` — Dynamic Catchall

- Looks up `slug` via `getPublishedPostBySlug()` (`lib/posts.server.ts`) against the `posts` table — there is no separate `dynamic_pages` table
- Renders the post's HTML `body` with `dangerouslySetInnerHTML`
- Falls back to 404 (`notFound()`) if no published post matches

Layer 5: Footer (`components/ChurchFooter.tsx`)

Displayed on every page:

- **Contact info** — Address, phone, email
- **Social links** — YouTube, Facebook, Instagram
- **Quick links** — Common pages
- **Copyright** — Auto-updates year
- **Accessibility statement**

Routing & Pages

Development-only Routes

- `/dev/blocks` — Gallery of every registered content block, rendered through the real serialise → parse → render pipeline rather than by importing the components directly, so it exercises the same path a live post does. Useful when building a block: no need to create a draft post to look at it.
- `/dev/blocks/editor` — Harness that mounts the real editor with the real Blocks sidebar and inspector, with no auth, no database and no save. It exists because `/login` is gated by Cloudflare Turnstile, which makes the editor UI hard to exercise otherwise. Shows the stored HTML and a live public render beneath the editor, so serialisation problems are visible as you type, and exposes the TipTap instance as `window.editor` for console work.

Both call `notFound()` when `NODE_ENV === "production"`. They mount admin UI without an auth check, so they must never be reachable on the live site.

Public Pages (No Auth Required)

Route	File	Purpose
`/`	`app/page.tsx`	Home page — hero, featured sermon, CTAs
`/about`	`app/about/page.tsx`	About church, team, vision, mission
`/beliefs`	`app/beliefs/page.tsx`	Statement of faith, doctrine
`/sermons`	`app/sermons/page.tsx`	Latest message as video (with an audio switch) + searchable podcast archive
`/sermons/[id]`	`app/sermons/[id]/page.tsx`	Individual sermon — YouTube embed, skip-to-sermon, next steps
`/live`	`app/live/page.tsx`	Livestream page — standard hero + section rhythm, with a client island that swaps between the custom glass player and an off-air card
`/contact`	`app/contact/page.tsx`	Contact form, address, hours
`/give`	`app/give/page.tsx`	Giving info — bank details, online giving
`/shop`	`app/shop/page.tsx`	Store front — published products grid with category filter chips (`ShopProductGrid`), editorial `/links` style
`/shop/[slug]`	`app/shop/[slug]/page.tsx`	Product detail — gallery, size/colour variant picker, add to basket
`/shop/cart`	`app/shop/cart/page.tsx`	Basket (client, zustand + localStorage)
`/shop/checkout`	`app/shop/checkout/page.tsx`	Contact details + embedded Stripe Payment Element
`/shop/checkout/success`	`app/shop/checkout/success/page.tsx`	Order confirmation; clears the basket
Also served at `shop.destinytees.uk/*`	`next.config.ts` `rewrites()`	Legacy subdomain host-rewrites to `/shop/*`
`/visit`	`app/visit/page.tsx`	First-time visitor guide, parking, service times
`/new-here`	`app/new-here/page.tsx`	Onboarding for new members
`/hire`	`app/hire/page.tsx`	Venue hire enquiry form
`/serve`	`app/serve/page.tsx`	Volunteer opportunities
`/connect`	`app/connect/page.tsx`	Connect groups (small groups)

Route	File	Purpose
`/kids`	`app/kids/page.tsx`	Kids ministry (ages 0-11)
`/youth`	`app/youth/page.tsx`	Youth ministry (ages 11-18)
`/young-adults`	`app/young-adults/page.tsx`	Young adults (18-30s)
`/missions`	`app/missions/page.tsx`	Mission partners, outreach
`/alpha`	`app/alpha/page.tsx`	Alpha course info, next event
`/bible-course`	`app/bible-course/page.tsx`	The Bible Course (Bible Society), next event
`/cap-money`	`app/cap-money/page.tsx`	CAP Money Course (Christians Against Poverty), next event. CTAs fall back to `/contact` when nothing is scheduled
`/whats-on`	`app/whats-on/page.tsx`	Events listing — featured-event banner, then upcoming events grouped by month
`/whats-on/[slug]`	`app/whats-on/[slug]/page.tsx`	On-site event page — one per ChurchSuite *series*, with all upcoming sessions, sanitised description, map link, signup and .ics
`/home`	`app/home/page.tsx`	**Temporary** event-card variant preview of the homepage (`?card=a|a-pill|c`). noindex — delete once a variant is chosen
`/whats-on/new`	`app/whats-on/new/page.tsx`	**Temporary** event-card variant preview of What's On. noindex — delete once a variant is chosen
`/connect-card`	`app/connect-card/page.tsx`	Prayer requests, connection form
`/jobs`	`app/jobs/page.tsx`	Job listings
`/jobs/[slug]`	`app/jobs/[slug]/page.tsx`	Job detail page
`/training`	`app/training/page.tsx`	`/training` resource library — category → subgroup (optional password) → post
`/baptism`	`app/baptism/page.tsx`	Baptism sign-up
`/child-dedication`	`app/child-dedication/page.tsx`	Child dedication request
`/volunteer`	`app/volunteer/page.tsx`	Volunteer sign-up form
`/help`	`app/help/page.tsx`	Help centre / FAQ
`/links`	`app/links/page.tsx`	"Next Steps" link-in-bio style page

Route	File	Purpose
`/nfc`	`app/nfc/page.tsx`	"Digital back of seats" — what an NFC tag or QR code on a seat opens during a service. Standalone (no header, footer, site popup or smart search) and `noindex`. Connect Card and Giving are hardcoded fixtures; everything else comes from `nfc_tiles`, including event tiles that resolve against the live ChurchSuite feed and hide themselves once the event has run
`/destiny-recovery`	`app/destiny-recovery/page.tsx`	Recovery course info page
`/dckids`	`app/dckids/page.tsx`	Destiny Kids Camp 2026 campaign page
`/accessibility`	`app/accessibility/page.tsx`	Reduced-motion / glass-FX preferences (client component)
`/privacy`	`app/privacy/page.tsx`	Privacy policy
`/terms`	`app/terms/page.tsx`	Terms of use
`/safeguarding`	`app/safeguarding/page.tsx`	Safeguarding policy
`/data-gdpr`	`app/data-gdpr/page.tsx`	Data & GDPR policy
`/governance`	`app/governance/page.tsx`	Transparency page: charity + company registration details, trustees/directors, charitable objects, five-year financial history and filings. Data comes live from the Charity Commission and Companies House APIs via `lib/governance.server.ts`, cached weekly (`revalidate = 604800`), falling back to a stored snapshot per-regulator when a key is missing or an API is down
`/admin-login`, `/administration`	`app/admin-login/page.tsx`, `app/administration/page.tsx`	Stale-bookmark redirects to `/login` and `/admin`
`/auth/callback`, `/auth/confirm`	`app/auth/callback/route.ts`, `app/auth/confirm/route.ts`	Supabase OAuth callback and email-OTP verification route handlers
`/[slug]`	`app/[slug]/page.tsx`	Dynamic catchall — looks up a published row in the `posts` table. At ≥1600px viewport the layout becomes a three-track grid with a promo card in each margin (see

Route	File	Purpose
		`posts/PostRails.tsx`); the 768px article column is unchanged at every width

Admin Pages (Auth Required, Checked in `middleware.ts`)

All admin/staff features live under a single `/admin` prefix with one login at `/login`. Each section requires a specific access-level role (see Authorization Layers); Super Admins get everything. `/admin/hr` is built but intentionally unlinked from any nav (not yet launched; Super Admin only).

Route	File	Purpose
`/login`	`app/login/page.tsx`	Staff sign-in
`/admin/forgot-password`	`app/admin/forgot-password/page.tsx`	Password reset request
`/admin/reset-password`	`app/admin/reset-password/page.tsx`	Password reset form
`/admin`	`app/admin/page.tsx`	Admin dashboard home
`/admin/banner`	`app/admin/banner/page.tsx`	Manage site banners
`/admin/popup`	`app/admin/popup/page.tsx`	Manage pop-ups
`/admin/redirects`	`app/admin/redirects/page.tsx`	Manage URL redirects
`/admin/cache`	`app/admin/cache/page.tsx`	Invalidate ISR cache
`/admin/posts`	`app/admin/posts/page.tsx`	Standalone content pages
`/admin/training`	`app/admin/training/page.tsx`	Training categories → subgroups → posts
`/admin/alpha`	`app/admin/alpha/page.tsx`	Manage Alpha **and Youth Alpha** events — wrapper over `CourseAdminPage`
`/admin/bible-course`	`app/admin/bible-course/page.tsx`	Manage The Bible Course events — wrapper over `CourseAdminPage`
`/admin/cap-money`	`app/admin/cap-money/page.tsx`	Manage CAP Money Course events — wrapper over `CourseAdminPage`
`/admin/recovery`	`app/admin/recovery/page.tsx`	Manage Recovery course events — wrapper over `CourseAdminPage`
`/admin/featured-course`	`app/admin/featured-course/page.tsx`	Choose the What's On featured course
`/admin/featured-event`	`app/admin/featured-event/page.tsx`	Promote one ChurchSuite event — picker plus headline/blurb/image/CTA overrides and a promote window
`/admin/event-popup`	`app/admin/event-popup/page.tsx`	Copy for the popup advertising the featured event (writes `popup_*` on the same row)
`/admin/nfc`	`app/admin/nfc/page.tsx`	Tiles on the `/nfc` page — add/edit/reorder/hide. A ChurchSuite form embed, artwork + copy + CTA, or

Route	File	Purpose
		an event picked from the live calendar (events without a framable signup are shown disabled with the reason)
`/admin/hr`	`app/admin/hr/page.tsx`	HR dashboard (staff, leave, jobs, documents, reviews) — unlinked, in progress
`/admin/store`	`app/admin/store/page.tsx`	Store — product list
`/admin/store/products/new`	`app/admin/store/products/new/page.tsx`	Create a product (name → editor)
`/admin/store/products/[id]`	`app/admin/store/products/[id]/page.tsx`	Product editor — details, photos, size/colour variants, stock
`/admin/store/hero`	`app/admin/store/hero/page.tsx`	Shop hero slides — add/edit/reorder rotating hero
`/admin/store/orders`	`app/admin/store/orders/page.tsx`	Orders list
`/admin/store/orders/[id]`	`app/admin/store/orders/[id]/page.tsx`	Order detail — mark fulfilled/cancelled/refunded
`/admin/users`	`app/admin/users/page.tsx`	Manage admin logins and their access-level roles (Super Admin only)

Components

Global Components

`ui/Button.tsx`

Client-safe, no directive. The site's one button. Before it existed there was no UI layer at all: every button was a hand-written Tailwind string, and a survey found ~60 variations of the orange pill alone — same intent, drifting padding and shadow.

```
<Button href="/contact" size="lg">Find out more</Button>
<Button variant="onDark" onClick={openSignup}>Sign up</Button>
```

Prop	Values	Notes
`variant`	`primary` · `secondary` · `outline` · `onDark` · `glass`	`onDark` / `glass` are for hero photography
`shape`	`pill` (default) · `soft`	`soft` is `rounded-xl`, the `/admin` chrome
`size`	`xs` · `sm` · `md` · `lg` · `xl`	`sm` is the `/admin` default
`href`	string	Renders `next/link`; `http(s):` / `mailto:` / `tel:` get an external anchor
`fullWidth`	boolean	

Every size is a padding cluster that already existed in the codebase (`px-6 py-3` appeared 14×, `px-7 py-3` 13×, `px-6 py-2.5` 13×), so adopting it does not shift anything by a few pixels. It also adds focus-visible rings and disabled states, which most of the hand-written buttons lacked.

> **Do not override padding through `className`.** `lib/cn.ts` is a plain join with no Tailwind conflict resolution (no `tailwind-merge`), so a `px-7` beating a size's `px-6` depends on stylesheet order rather than anything guaranteed. Add a size instead.
>
> Genuinely one-off buttons should stay plain `<button>` elements — this is for the repeated cases. Migration is opportunistic; most call sites are still hand-written strings.

`ChurchHeader.tsx`

- **What:** Site navigation header
- **Props:** None (uses client context for mobile menu state)
- **Behavior:**
- Desktop: Horizontal menu bar with search
- Mobile: Hamburger menu (drawer slides from left)
- Active route highlighting
- Search filters sermons by title/speaker (client-side fuse.js)

`ChurchFooter.tsx`

- **What:** Sitewide footer (server component — awaits `isYouTubeQuotaExceeded()` to drop the Sermons link when the YouTube quota is blown)
- **Displays:** Brand blurb + address, three link columns (Church / Connect / Legal), copyright, Report a Bug, phone
- **Layout:** 4-column grid from `md:` up; on mobile the three link columns render as accordions via `FooterLinkGroup`

`FooterLinkGroup.tsx`

- **What:** One footer link column — a client component so it can hold open/closed state
- **Mobile:** Collapsed accordion with a chevron header (~44px tap targets), so the footer isn't ~23 stacked links
- **Desktop:** Forced open with CSS only (`md:grid-rows-[1fr]`, `md:pointer-events-none`, chevron wrapper `md:hidden`) — no JS media query, so no hydration mismatch

- **Why it matters:** Links stay in the DOM at every breakpoint (SEO/crawl-safe). Collapsed panels also get `invisible` so hidden links leave the keyboard tab order
- **Gotcha:** The chevron's `md:hidden` lives on a plain wrapper `<span>` — the global `.material-symbols-rounded` rule in `app/globals.css` overrides Tailwind display utilities applied directly to the icon

`CookieBanner.tsx` + `lib/cookieConsent.tsx`

- **What:** GDPR cookie consent, split into **three categories** rather than a single on/off:
- `essential` — always `true` , keeps the site running
- `media` — third-party embeds: ChurchSuite forms, and YouTube in the live/sermon/missions players
- `analytics` — Vercel Analytics
- **State:** `CookieConsentProvider` (a React context, hook `useCookieConsent`) holds the choice and persists it to `localStorage` under `destiny-cookie-consent` . `decided` is `true` once any choice is stored.
- **Banner behavior:**
- Renders only after mount and only while no choice is stored (hydration-guarded so SSR/client markup match)
- Two buttons: **"Accept all cookies"** (`allowAll` → media + analytics on) and **"Necessary cookies only"** (`denyOptional` → both off). Both link out to the Privacy Policy and Terms of Use
- A finer `savePreferences({ media, analytics })` also exists for per-category control
- **Gotcha:** Because consent loads from `localStorage` in an effect, `consent` is `null` on first render. Every gated component waits for `mounted` before reading it, so nothing embeds or tracks until the stored choice (or its absence) is known.

`AnalyticsGate.tsx`

- **What:** Conditional analytics loading
- **Logic:** Renders `<Analytics />` (Vercel) only when `consent?.analytics` is `true` ; returns `null` otherwise

Media-gated embeds (`consent?.media`)

`ChurchSuiteEmbed.tsx` , `live/LivePlayer.tsx` , `sermons/SermonPlayer.tsx` and `missions/MediaEmbed.tsx` all gate their third-party `<iframe>` on `consent?.media === true` . Until media cookies are accepted they render an in-place placeholder ("Cookies required to load this form/video") with an **Accept all cookies** action wired to the same `useCookieConsent` context, so a visitor can opt in without scrolling back to the banner.

Once consent is in, `ChurchSuiteEmbed` and `MediaEmbed` cover the iframe with `ui/EmbedLoadingOverlay` until it fires `onLoad` — see below.

`ui/EmbedLoadingOverlay.tsx`

- **What:** The spinner-and-copy layer over a third-party iframe that hasn't painted yet. Used by `ChurchSuiteEmbed` (forms, light tone) and `MediaEmbed` (video, dark tone).
- **Why it escalates:** a cross-origin iframe gives us exactly one signal, `onLoad` — there is no progress to report. A single static "Loading" sitting there past a few seconds reads as *broken* rather than slow, and the visitor closes the modal. On `/give` , `/connect` and the `/nfc` tiles that is the whole conversion. So the line changes as time passes: movement is what says something is still happening.

- **Stages:** `Loading` → `Still loading` (3s) → `Taking longer than usual` (8s) → `Thanks for waiting` (14s), the last one alongside an **Open the form in a new tab** link. Timings live in `lib/embedLoading.ts` and are pinned by `tests/unit/embed-loading.spec.ts`.
- **The copy is true at the point it appears**, deliberately. Invented machinery ("Connecting to secure server", "Encrypting your details") holds attention *because* it claims things the site is not doing, and half these embeds are the giving form — being caught inventing a security step on a donation page costs more than the bounce it saves. The escalation does the work without the claim.
- **Props:** `loaded` (drives the fade), `tone` (`light` | `dark`), `fallbackHref` / `fallbackLabel` (omit to leave the escape hatch out — worth having for a form the visitor came to fill in, less so for a video).
- **A11y:** the message is a `role="status"` live region — "taking longer than usual" is the one thing a screen-reader user cannot otherwise get from an iframe that hasn't painted. The spinner is `aria-hidden`. On load the overlay fades and *then* flips to `visibility: hidden` (stepped transition in `globals.css`), which takes the stale copy out of the accessibility tree and the fallback link out of the tab order without cutting the fade short.

`SiteBanner.tsx`

- **What:** Top-of-page announcement banner
- **Data:** `site_banner` table (fetched in root layout)
- **Features:**
- Multiple banner types (sitewide, alpha events, recovery)
- Optional CTA link
- Can be dismissed by user (session storage)

`SitePopup.tsx`

- **What:** Modal pop-up overlay
- **Data:** `site_popup` table
- **Features:**
- Shows once per session (if `show_once=true`)
- Title, body (markdown), image, CTA button
- Dismiss button
- Centered on screen

`WelcomeWidget.tsx` — removed; merged into `FloatingSmartSearch`

The "New here?" script is no longer a separate widget. Two floating controls in two corners was two things to notice; there is now one. See `FloatingSmartSearch.tsx` above — the script lives there as its `welcome` mode, and the copy still lives in `lib/welcomeChat.ts`.

`shop/ShopHero.tsx`

- **What:** Dynamic, auto-rotating hero at the top of `/shop`
- **Data:** `shop_hero_slides` table (via `getActiveShopHeroSlides()`; passed in as a prop from the server component)
- **Features:**
- Image-backed slides (next/image `fill`) with dark gradient + Anton headline + orange CTA pill
- Auto-rotates (~6s crossfade) with 2+ slides; static with one

- Pauses on hover/focus; honours `prefers-reduced-motion` (no auto-advance)
- Keyboard-navigable dot indicators
- Falls back to the static "The Destiny Store" masthead when no slides are active (handled in `app/shop/page.tsx`)

`FloatingSmartSearch.tsx`

- **What:** the site's single floating widget. Two ways in from one control: the AI conversational search, and a hand-written **"New here?" script** for visitors who don't yet know what to ask.
- **Two modes.** `mode: "search" | "welcome"` . Search is the AI thread (`/api/chat`); welcome walks the tree in `lib/welcomeChat.ts` with no network call at all. Threads are held in separate state (`messages` vs `welcomeTurns`) so a scripted reply can never render into a streaming one. Typing a question while the script is open switches to search and drops the script — one thread at a time.
- **The teaser.** The resting control alternates between two faces on an even **10s / 10s** split (`TEASE_PHASE_MS`): the familiar circle with the search mark, then the sparkle with the pill grown to **13.5rem** reading "New here? Start here". Clicking it in that state opens the script; clicking the search mark opens the AI bar as before. This is the only advertisement the guided path gets, since nothing auto-opens.
- **The icon turn is 2D on purpose.** `.fs-flip` cross-fades its two `.fs-flip-face` children with `opacity` + `scaleX` , not a `preserve-3d` card with `backface-visibility: hidden` . The flip lives inside `.glass` , whose `backdrop-filter` flattens the 3D rendering context; Safari then painted both faces mirrored on top of each other and the icon looked broken. Nothing about the effect needs real 3D, so the 2D version renders identically everywhere.
- **Reduced motion:** no cycling at all — something that changes width every few seconds is exactly the unsolicited movement `prefers-reduced-motion` asks us not to make. Those visitors get the label **permanently** instead, so they lose the motion without losing the discoverability.
- **Three widths, transitioned not keyframed.** `--fs-w` holds the resting width per state (circle `3.5rem` → teaser `13.5rem` → pill `min(100vw-2rem, 28rem)`) with `transition: width` . It was keyframes while there were only two states; three made the pairs combinatorial, and the keyframes sat on the same `animation` property as `.search-glow` 's loading spin **on the same element**, so a morph mid-request stopped the glow rotating. A transition frees `animation` for the glow and doesn't run on first paint, which is what the old transient `.morph-*` classes existed to prevent.
- **`searchEnabled` prop.** The widget is now **always mounted**; the prop only governs the AI half. The guided script is hand-written and has no service to be down, so the Smart Search kill-switch must not take it with it. With the prop false the input form is absent and the trigger opens the script directly.
- **Feature:** the AI half runs if the `smart_search` service is enabled
- **Behavior:**
- Click button → pill expands, chat thread opens
- Before the first message of a session, silently runs an **invisible Cloudflare Turnstile** challenge (`size: "invisible"` , `execution: "execute"`) and posts the token to `POST /api/turnstile/verify` , which sets a signed `ts_verified` cookie (see Authentication & Authorization). If the invisible check can't silently confirm the visitor, a **visible fallback widget** renders inline in the chat panel; solving it verifies and auto-resends the pending message. Skipped entirely once `ts_verified` is present and unexpired.
- User types query → full history sent to `/api/chat` (OpenAI, `gpt-4.1-mini`); the request is rejected (403) if `ts_verified` is missing/expired.

- The route uses **tool-calling** and streams **NDJSON** events (`text` , `tool_call` , `tool_result` , `done`). Prose is still parsed for the trailing `OPTION:` / `PAGE:` / `CTA:` lines (clarifying chips + a navigation CTA). Each `tool_call` event shows a short status line ("Searching the web…", "Reading the page…", etc.) while that tool runs.
- **Tools** (`lib/smartSearch/tools.ts`): `find_products` (searches published shop products via `getPublishedProducts()` + fuse.js, returns cards), `get_weather` (Open-Meteo, no key), `get_directions` (Google Maps embed), `search_web` (Tavily search, `search_depth: "advanced"`), `extract_page` (Tavily extract — reads the full content of one URL a prior `search_web` call actually returned; rejects any URL not in that request's `seenUrls` set, so the model can't be steered into fetching arbitrary pages).
- **Result cards** (`components/smartSearch/ResultCards.tsx`) render below the prose in Smart Search's glass style. The product card offers inline size/colour selection and **add-to-cart** (via `useCart()`), so a visitor can buy without leaving the conversation.
- Chat history + cards stored in component state (not persisted)
- **Optional env vars:** `NEXT_PUBLIC_GOOGLE_MAPS_EMBED_API_KEY` (directions embed — degrades to an "Open in Maps" link without it) and `TAVILY_API_KEY` (web search + page extraction — degrades to a "not configured" note). Weather and product search need no extra key. `NEXT_PUBLIC_TURNSTILE_SITE_KEY` / `TURNSTILE_SECRET_KEY` gate the widget; without them Turnstile verification is skipped client-side but the server fails closed (see below).

> **`VisualEditOverlay.tsx` does not exist.** There is no in-page visual editing overlay — it was part of the removed page-builder/Studio feature (migration `20260711_06_remove_page_builder.sql`). Content is now edited through dedicated admin forms (`/admin/posts` , `/admin/store/products/[id]` , etc.), each using `RichTextEditor.tsx` for rich text where applicable.

`GlassBloomTracker.tsx`

- **What:** Performance tracking script
- **Purpose:** Tracks bloom/glass effect performance for optimization

`LiveBanner.tsx`

- **What:** "WE ARE LIVE" banner bar, styled like `SiteBanner.tsx` 's bars
- **Data:** `LiveContext` (server-seeded in root layout via `getLiveStatus()` , then polled client-side every 30s)
- **Behavior:** Renders at the top banner slot (offsetting any DB banner below it) whenever the channel is live; hidden on `/live` and `/admin/*` . CTA links to `/live` .

`contexts/LiveContext.tsx`

- **What:** The single client-side source of live state, consumed by the banner and every part of `/live`
- **Seeding:** The root layout passes `getLiveStatus()` straight in, so the first paint is already correct — polling only ever corrects it afterwards
- **Polling:** `/api/youtube/live` every 30s, plus an immediate poll on mount and on `visibilitychange` / `focus` / `online` . The mount poll matters: the server render can be a minute stale, and "we went live 40 seconds ago" is exactly when someone opens the page.

- **Grace period:** `live` does not drop to false until **two consecutive** negative polls (`OFFLINE_GRACE`), so one flaky request doesn't pull a running service off someone's screen. The streak is counted per poll — an earlier version counted it in an effect keyed on `live`, which only re-runs when the boolean flips, so it could never reach two.
- **`markOffline()` :** lets the player tear the live view down immediately on its ENDED/error event, well before YouTube's own pages agree

`/live` components (`components/live/*`)

The page itself is a server component carrying the site's normal hero + alternating `bg-white` / `bg-[#f5f7fa]` sections (`AnimateIn` reveals, `font-black` headings, orange eyebrows, `WatchOnYouTubeBand` , `WorshipWithUsSection`). Only the parts that change with the broadcast are client islands:

- **`LiveStage.tsx`** — the player when we're on air, an off-air card the rest of the week. Live: red "On air now" eyebrow, the broadcast title (splitting the `Title || Ps Speaker` upload convention), start time, and an "Open on YouTube" escape hatch. Off air: next-service countdown, links to `/sermons` and `/visit` , and the latest message as a thumbnail card.
- **`LiveHeroStatus.tsx`** — the hero eyebrow. A red pulsing "Live now" pill while streaming, the site's standard orange eyebrow otherwise.
- **`useLiveNow.ts`** — the one place the "are we live?" rule is derived, so the hero badge and the player can't disagree.
- **`NextServiceCountdown.tsx`** — "Sunday 14 September, 11:00am · in 2 days 4 hours". The date renders on the server too (it's identical either side of hydration); the relative half waits for the client, since a countdown computed server-side is wrong by however long the response sat in a cache.
- **`LivePlayer.tsx`** — YouTube IFrame API player with `controls=0` and a fully custom glass control bar (play/pause, mute, volume, fullscreen, live-edge seek). See `lib/youtubeIframe.ts` for the shared API loader (also used by `SermonPlayer.tsx`). `onEnded` is held in a ref so the mount effect stays keyed on the video id alone, and `onError` is treated as an ending too — a pulled or privated broadcast otherwise leaves the player wedged on a black rectangle.

Page-Specific Components

Ministry pages (`components/ministry/*`)

The shared vocabulary for the five ministry pages — `/child-dedication` , `/kids` , `/youth` , `/young-adults` and `/connect` . Before this existed each page carried its own byte-identical copy of the hero and the feature-card grid, so the five were visually interchangeable and a change meant editing five files. All server components.

- `MinistryHero.tsx` — the page hero. Sharp photo through `next/image` `fill` + `priority` + `sizes="100vw"` (the previous heroes used a blurred CSS `background-image` , which could be neither optimised nor prioritised as the LCP element). Responsive `min-h` , centred content, and a `chips` prop rendering `glass glass-sm glass-pill` fact pills — those chips are what differentiate the five heroes, and they're where `/kids` and `/youth` carry their schedule instead of cramming it into the eyebrow. `objectPosition` frames per-page (`/kids` needs `top`).
- `SplitSection.tsx` — image-and-copy split on a 7/5 asymmetric 12-column grid rather than an even 50/50. `reverse` swaps columns via `lg:order-*` so JSX stays in reading order; `wideMedia` flips the ratio

for photo-led sections; `tone="dark"` renders the shared `#363f48 → #242e37` band.

- `ImageMosaic.tsx` — 3-or-4 image offset mosaic replacing the old `col-span-2` -plus-two-squares collage. Every tile is `fill` inside a fixed aspect box with a real `sizes`, which is what removes the layout shift and oversized downloads the old fixed-`width` / `height` collages caused.
- `FeatureGrid.tsx` — the "why join / what we're about" cards. Card shell borrowed from `events/EventCard` (hairline border, two-layer shadow, hover lift); the orange-tinted icon square was dropped in favour of an oversized index numeral. One `AnimateIn` wraps the whole grid — `AnimateIn` renders a plain `<div>`, so wrapping cells individually would break the grid.
- `AgeGroupCards.tsx` — age-banded groups (`/kids` classes, `/youth` key stages), which previously had two different designs for the same job. Colours come from the `destiny-*` theme tokens, not the inline hex strings the pages used to carry.
- `FactStrip.tsx` — when/where/cost as one divided panel instead of a third identical card row.
- `FormSection.tsx` — the dark ChurchSuite band on `/child-dedication` and `/connect`. Wraps `ChurchSuiteEmbed` in a white panel; the embed keeps its own cookie-consent gate untouched.

Kids Camp (`components/kids/*`)

Section components for `/dckids` (Kids Camp). Separate from `components/ministry/*` — this page has not been brought onto the shared vocabulary yet.

- `KidsCampHero.tsx`, `KidsCampDetails.tsx`, `KidsCampTeam.tsx` (safeguarding leads), `KidsCampVideo.tsx`, `KidsCampForm.tsx`, `KidsCampFAQ.tsx` (client-side accordion)

Governance (`components/governance/*`)

Section components for `/governance`. All are server components taking plain props from `getGovernanceData()`, and each returns `null` when it has nothing verified to show — an absent section is better than an empty heading on a transparency page.

- `GovernanceHero.tsx` — eyebrow, title, lede
- `RegistrationCards.tsx` — paired charity/company cards with numbers, status, dates, registered office and outbound links to each official register
- `CharitableObjects.tsx` — charitable objects, activities and areas of operation
- `TrusteesDirectors.tsx` — trustee and director lists. Deliberately name-and-role only: both APIs also return dates of birth and correspondence addresses, which are not republished here
- `FinancialHistory.tsx` — headline income/expenditure plus per-year bars scaled against the largest figure across all years, so years stay comparable to each other
- `FilingHistory.tsx` — recent Companies House filings (linked to the official documents) and charity annual-return submission dates
- `GovernanceSourceNote.tsx` — attribution, refresh cadence, and an explicit notice naming any regulator whose data is being served from the stored snapshot rather than live

Events (`components/events/*`)

The one place event UI lives, shared by What's On, the homepage carousel and the preview routes. Before this existed the card was duplicated three times (`whats-on/EventsGrid`, `home/WhatsOnSection`, `posts/PromoRail`) and the copies had drifted.

- `EventCard.tsx` — **the** event card, in three trial variants selected by a `variant` prop: `a` (restrained: neutral surface, hairline border, orange kicker + CTA only — the default that ships live), `a-pill` (`a` plus a category chip on the artwork), `c` (full-bleed poster, copy on a `.glass` panel). Variants `a-pill` and `c` exist only for the preview routes; delete the unused bodies once a variant is chosen. Server-safe (no `"use client"`, no hooks) and — importantly — **never reads the clock**: filtering and month grouping happen server-side, or the client would hydrate a different set of events than the server rendered at midnight and DST boundaries.
- `EventCardArtwork.tsx` — artwork plus the two no-artwork fallbacks. Roughly half the feed has no image, so the fallback is a first-class state (large date numeral, or a brand panel for `c`).
- `EventsGrid.tsx` — the What's On grid. Receives month groups pre-filtered and pre-sorted from the server and does **only** the text search. Sticky month headings; keeps `id="events"`, which ChurchHeader, the footer and the sitemap all deep-link to.
- `FeaturedEventHero.tsx` — the wide banner above the grid. Takes an *already-merged* `ResolvedFeaturedEvent`, so it holds no fallback logic and renders `null` when nothing is live. Sits deliberately **outside** the `#events` section so that anchor still lands on the grid.
- `EventPopup.tsx` — the featured-event popup. `sessionStorage` (once per visit, unlike SitePopup's `localStorage` once-per-visitor), keyed on `identifier:updated_at` so changing the event or editing the copy re-shows it. Suppressed on `/whats-on` and the event's own page via `usePathname`, because the root layout is a server component and cannot know the path.
- `EventSignupButton.tsx` — the event page's primary CTA. Opens the ChurchSuite event **in a modal** (`AlphaSignupModal` at `size="lg"`) instead of a new tab, so the whole multi-step signup — ticket picker, details form, confirmation — completes without leaving the site. ChurchSuite's own event pages frame fine and submit over AJAX. Third-party ticket URLs (Eventbrite and friends) send `X-Frame-Options: DENY`, so any non-`churchsuite.com` host keeps the old new-tab link. When the event takes signups the embed loads at `#form_event_signup`, so it opens scrolled past the artwork, dates, description and map that the page around the modal already shows. A fragment is the only lever available — the iframe is cross-origin, so its DOM is opaque to our CSS and JS, and none of ChurchSuite's URL variants (`/embed/events/…`, `/events/…/signup`, `?embedded=1`) serve a signup-only view; all return the identical full page. Events with signups closed have no such element and open at the top, which is the right fallback.

`AlphaSignupModal.tsx` / `ChurchSuiteEmbed.tsx`

The one ChurchSuite-in-a-modal treatment, used by the course pages and now by every event. `size="lg"` gives a `h-[88vh] max-w-3xl` panel whose embed **fills** it (`ChurchSuiteEmbed fill`) rather than taking a fixed height: an event page is arbitrarily long, and a fixed height would mean either dead space or two nested scrollbars. The default `md` size keeps the fixed 620px box the course forms were built against.

`PopupShell.tsx`

Shared modal chrome (backdrop, close button, image, title, body, CTA, `z-[100]`), extracted from `SitePopup` so it and `EventPopup` cannot drift visually. The *behaviour* around it stays with each caller — including the open delay: both `SitePopup` and `EventPopup` wait **7 seconds after load** (`setTimeout(..., 7000)`) before showing, so a visitor sees the page before a modal covers it. **An active event popup suppresses the site popup**: `app/layout.tsx` passes `null` to `<SitePopup>` whenever `getActiveEventPopup()` resolves, so only ever one modal shows and there is no client-side coordination to get wrong.

These two are the only self-opening surfaces on the site. `FloatingSmartSearch` shares the screen with them but needs no coordination, because it never opens itself — an earlier auto-opening version did, and carried a `lib/popupGate.ts` zustand flag that both popups checked before arming their timer. That gate was deleted along with the overlay.

Admin Components (`components/admin/*`)

- `AdminSidebar.tsx` — Admin navigation menu
- `AdminHeader.tsx` — Sticky desktop header for the admin shell; shows an "Admin / {section}" breadcrumb (title derived from the pathname) and a "View live site" button
- `RichTextEditor.tsx` — Shared TipTap rich-text editor (HTML output); used by posts, training posts, HR job descriptions, and (since `a22301b`) shop product descriptions. Optional `blocks` / `onEditor` props admit content blocks — schema and drop handling only; the blocks UI is a separate surface owned by the parent, deliberately **not** part of this toolbar. The toolbar does exactly one job: reformat the current text selection. The old "Embed YouTube video", "Embed ChurchSuite form" and "Embed HTML" buttons are now the Video, ChurchSuite form and Custom embed blocks — they inserted new objects rather than formatting a selection, so they belonged in the sidebar. `enableYouTube` and `enableHtmlEmbed` now only register the legacy `youtube` / `htmlEmbed` nodes so pages authored with those buttons keep parsing; `enableChurchSuite` is gone entirely (it only ever drove a button, since ChurchSuite embeds were stored as `htmlEmbed`).
- `blocks/*` — **Client.** Editor side of the content-block system: the TipTap node factory, node-view chrome, the Blocks sidebar, the schema-driven settings inspector and its field components. See Content Blocks.
- `ChurchSuiteEventFill.tsx` — **Client.** Collapsible "Fill from ChurchSuite event" picker embedded in the "Add Event" form of every course admin page. Fetches the same picker feed as `/admin/featured-event` (`/api/admin/events`) and, on selection, calls `onFill({ startDate, location, signupUrl, name })` to prefill those three form fields — no new table or API route. Deliberately leaves `format` / `frequency` /meeting fields untouched, since those `alpha_events` columns have no ChurchSuite equivalent.
- `CourseAdminPage.tsx` — **Client.** The single implementation behind `/admin/alpha` , `/admin/recovery` , `/admin/bible-course` and `/admin/cap-money` . See below.

Course admin pages

The four course pages used to be four copies of the same ~630-line file, differing only in slug, accent colour and wording — `bible-course` and `cap-money` were 627 lines each and differed by 84. Adding CAP meant copying one wholesale, and a fix to the event list had to be applied four times with nothing in the code to say so. They are now four-line route files over `CourseAdminPage` , driven by `COURSE_ADMIN_PAGES` in `lib/courseEvents.ts` (2,620 lines across 4 files → 766 across 5).

```
// app/admin/cap-money/page.tsx — the whole file
import CourseAdminPage from "@/components/admin/CourseAdminPage";

export default function CapMoneyAdminPage() {
  return <CourseAdminPage page="cap-money" />;
}
```

To add a course: write the `alpha_events_type_check` migration, add a `COURSE_EVENT_META` entry (label, href, colour, hint, events label, banner message and link text), add a `COURSE_ADMIN_PAGES` entry (heading, blurb, accent, promote copy, and the types it owns), and create the four-line route file. `tests/unit/course-events.spec.ts` fails if a type has incomplete metadata, is owned by two pages, or is owned by none.

Notes worth knowing before changing it: - **A page can own several types.** `/admin/alpha` manages Alpha and Youth Alpha together, which is why `types` is a list; the create form only shows the Type selector when there is more than one, and each type gets its own banner card and event list. - **Accent colour is applied as inline `rgba()`**, not Tailwind opacity classes, because the accent is per-course data and cannot be a compile-time class name. `hexToRgb` reproduces the `ACCENT_RGB` constants the pages used to hardcode. - **Headings are stored verbatim, not derived** — the article does not follow a rule ("Promote Destiny Recovery", "Promote The Bible Course", "Promote the CAP Money Course"). - **The routes stayed put.** Keeping the four URLs as wrappers rather than moving to a `[type]` route means no redirects, no `ROUTE_RULES` change and no sidebar change. - `tests/admin-courses.spec.ts` covers the rendered pages but is skipped unless `ADMIN_EMAIL` / `ADMIN_PASSWORD` are set.

> Redirects and banner management (`/admin/redirects` , `/admin/banner`) are built inline in their `page.tsx` files rather than as separate reusable components — there is no standalone `RedirectManager.tsx` / `BannerManager.tsx` / `MediaUploader.tsx` / `PageEditor.tsx` / `AdminSermonManager.tsx` . Sermon hiding was removed entirely (migration `20260711_07_remove_sermon_hiding.sql`), and the earlier page-builder/ Studio editor (`PageEditor.tsx` 's likely origin) was removed by `20260711_06_remove_page_builder.sql` .

Content Rendering (`components/content/*`)

- `RichContent.tsx` — **Shared (server-first).** Renders admin-authored rich text and upgrades any embedded content blocks into real React components. Replaces the bare `dangerouslySetInnerHTML` previously used on `/[slug]` , `/jobs/[slug]` , `/training/.../[postSlug]` , `/shop/[slug]` and `/whats-on/[slug]` . Content with no blocks takes the exact previous code path, so adopting it everywhere was a no-op. Also emits merged schema.org JSON-LD contributed by blocks. See Content Blocks.

Content Blocks (`components/blocks/*`)

- **Shared (no `"use client"`).** The blocks themselves — FAQ, callout, quote, card grid, gallery, buttons — plus the registry, wire-format serialisation and design tokens. These server-render with zero client JS on public pages and the same modules render inside the editor. See Content Blocks.

Public Training Components (`components/training/*`)

The member-facing pieces of the `/training` resource library.

- `PasswordGate.tsx` — **Client.** Shown in place of a sub-group's content until the correct password is entered; on success the server sets a signed unlock cookie (via `/api/training/unlock`) and the route refreshes to render the real content server-side.
- `CompleteButton.tsx` — **Client.** "Mark complete" control on a post. When `min_read_seconds > 0` it drives the HMAC read timer (`/api/training/posts/[id]/timer`): it `start` s a token on mount, counts down, and only allows completion once the server `verify` s the minimum read time.
- `CompletablePostList.tsx` — **Client.** Post list with a per-post completion indicator, kept in sync with the hero progress bar via the shared completed set.
- `TrainingProgress.tsx` — **Client.** Completion progress bar in the sub-group hero — shows how many of the group's posts are done.
- `useTrainingProgress.ts` — Per-browser completion store in `localStorage` (no per-user accounts; trainees share a group password). Exposes `useCompletedSet()` and `toggleCompleted()` ; starts empty

on first render to avoid hydration mismatch, then fills in after mount and stays reactive across tabs via a custom event + the `storage` event.

Admin Content/Training/HR Components (`components/admin/{posts,training,hr}/*`)

- `posts/PostEditor.tsx` — Standalone page editor (uses `RichTextEditor`). Full-screen at **both** breakpoints; only the panel placement differs. Desktop gets the permanent Blocks and Settings sidebars; mobile edits the title in the header, puts the slug and published switch behind a "Page settings" sheet, and gives the rest of the screen to the editor. It was previously a `Modal` on mobile — the whole form inside a scrolling popup, with the editor capped at 420px and scrolling separately inside that, so the page content got about a third of the screen and a newly added block was immediately pushed out of sight.
- `training/PostEditor.tsx` — Training post editor (uses `RichTextEditor`). Still a `Modal` on mobile: its body is one field among many rather than the whole point of the screen, and it inherits the mobile block sheets and toolbar from `BlockTools` either way.
- `training/CategoryModal.tsx`, `SubgroupModal.tsx`, `FolderModal.tsx`, `IconPicker.tsx` — Training tree CRUD modals
- `hr/HrUI.tsx` — Staff directory, leave requests, documents (main HR dashboard shell)
- `hr/JobModal.tsx` — Create/edit job listing (uses `RichTextEditor`)
- `hr/modals.tsx` — Remaining HR CRUD modals (staff, leave, reviews, applications)

Post Promo Rails (`components/posts/*`)

Desktop-only internal advertising in the empty margins of a post page. Added 2026-07-26.

- `PostRails.tsx` — **Server.** Exports `EventsRail` (left) and `CoursesRail` (right). Builds serialisable `PromoCard[]` and hands them to the client rotator. Events come from `fetchChurchSuiteEvents` (filtered to future dates and sorted — unlike `EventsGrid` / `WhatsOnSection`, which show past events too); courses come from `lib/courses.ts` with the admin-featured one first via `getFeaturedCourseId`.
- `PromoRail.tsx` — **Client.** Mounts every card stacked and shows one at a time, advancing every 15s. Pauses on hover/focus and does not auto-rotate under `prefers-reduced-motion` (WCAG 2.2.2). Card = date, artwork, Anton title, clamped description, CTA.
- `lib/promo-palette.ts` — Pure colour maths: 4-bit histogram dominant colour, reduced to an `"H S%"` pair. Near-black/near-white histogram buckets are skipped so cards don't all take the colour of a white studio backdrop.
- `lib/promo-palette.server.ts` — Fetches and decodes remote artwork (JPEG/PNG only), memoised per image URL. Cards with no usable artwork get `DESTINY_PALETTE` (brand orange).
- `scripts/precompute-course-palettes.ts` — Regenerates `COURSE_PALETTES` in `lib/courses.ts`. Run after changing any course `card.image`: `npx tsx scripts/precompute-course-palettes.ts`.

Card treatment. The card is light on purpose: a near-white tint of the sampled hue, a hairline border, dark text, and the sampled colour at full strength only in the CTA pill. One `"H S%"` pair drives all three (`hsl(${accent} 97%)` / `88%` / `30%`), so the whole lightness ramp is tunable in one place. An earlier version filled the card with a dark gradient, which made two 300×600 slabs the heaviest thing on the page and pulled the eye off the article — if you are tempted to add weight back, that is the trade being made.

Why the rotation has no animation. Cross-fading double-exposed two opaque text-bearing cards (both titles legible at 50%), and fading the incoming card up from `opacity: 0` left it stranded invisible when a background tab throttled the transition. The swap is instant and nothing transitions opacity. Unannounced motion in the page margins also competes with the article. The hover lift stays — that one is user-driven. All cards stay mounted so their images load once up front; mounting only the active card meant every rotation inserted a fresh lazy `<img>` that flashed blank. Inactive cards are `aria-hidden` with `tabIndex={-1}`.

> **Why no `sharp` here.** The first version used `sharp().stats().dominant`. That traced ~20MB of libvips into the `/[slug]` function and pushed it to 256MB, over Vercel's 250MB uncompressed limit — the build passed and the *deploy* failed. `jpeg-js` + `pngjs` are ~800KB combined and produce identical output. Course artwork is WebP, which neither decodes, so those four palettes are precomputed into `lib/courses.ts` instead — they're static images, so a literal is cheaper and safer than any runtime decode. **Do not import `sharp` into anything reachable from a page component.** It is fine in the admin upload routes, which are their own functions.

Layout lives in `app/[slug]/page.tsx`: below 1600px the page is byte-identical to before; at 1600px+ it becomes `grid-cols-[300px_minmax(0,768px)_300px]` with `gap-16`. The maths is `300 + 64 + 768 + 64 + 300 = 1496` of content, `+64` of `lg:px-8` = the `max-w-[1560px]` cap, which is why the breakpoint sits at 1600. **Every pixel of gutter costs two**, so changing the gap means recomputing the cap and the breakpoint together. The article is first in the DOM and placed with `col-start-2`, so promo content never precedes it for crawlers or screen readers. **Do not add `self-start` / `h-fit` to the rail `<aside>`** — the grid's default stretch is what gives the sticky card its scroll range; hugging the content silently disables sticky.

NFC Page (`components/nfc/*`)

The tiles on `/nfc` — the "digital back of seats" page an NFC tag or QR code on a seat opens.

- `NfcTileGrid.tsx` — the card grid. Cards are `<button>`s, not links: everything opens in place, because the page exists to be finished before the next song starts. Holds the open-tile state and the ref to the invoking card so focus can be restored on close. The `BADGE` lookup is what the card promises before you tap it — "Form", "Sign up", "Details".
- `NfcTileModal.tsx` — the popup. Three modes, two layouts: `embed` frames a ChurchSuite form (`ChurchSuiteEmbed`) with the "more details" link hung off the bottom, so the popup answers the question *and* the full page stays one tap away; `event` is the same layout with no branch of its own, because `getNfcTiles()` resolves an event tile's signup URL into `embedUrl` server-side; `info` shows artwork + copy + an orange CTA, the `PopupShell` layout. Unlike the four older copies of this modal (`AlphaSignupModal` , `ConnectCardCTAs` , `GiveCTA` , `YouSaidYesButton`) it has real dialog semantics — `role="dialog"` , `aria-labelledby` , focus in on open, focus restored on close, and a Tab trap — because `/nfc` is the one page used cold by people who have never been to the site. Its entrance is a CSS keyframe (`.nfc-modal-*` in `globals.css`) rather than the visible-state-plus-double-rAF the others use: closing unmounts immediately, so the entrance was the only transition that ever ran.

Chrome suppression. `/nfc` is deliberately standalone. Four components carry the path check: `ChurchHeader.tsx` , `FooterGate.tsx` , `FloatingSmartSearch.tsx` , and both popups (`SitePopup.tsx` , `events/EventPopup.tsx`). The popups matter most — an announcement auto-opening on top of a tile popup, mid-service, is the one failure this page can't afford. The cookie banner and site banners stay: they're consent and legal, not chrome.

Connect Card (`components/connect-card/*`)

- `ConnectCardCTAs.tsx` — Call-to-action buttons
- The prayer/connection form itself is built inline in `app/connect-card/page.tsx` , not a separate component

Report a Bug (`components/report-bug/*`)

- `ReportBugLink.tsx` — a "Report a Bug" link rendered in `ChurchFooter.tsx`. Opens an accessible in-app modal (Escape-to-close, body-scroll lock) with a small form: **Full Name**, **Email**, and **How to reproduce**. On submit it captures the current `window.location.href` as `pageUrl` and calls the `submitBugReport` server action, showing inline loading / success / error states.
- `actions.ts` — `submitBugReport(formData)` server action. Validates name/email/steps, then uses `GITHUB_TOKEN` to open a GitHub Issue on `SquareMediaGroup/destinychurch` via the GitHub REST API (`POST /repos/.../issues`), titled `Bug Report: <name>` with the reporter, page URL, and reproduction steps in the body. Returns `{ success, error? }`; a missing `GITHUB_TOKEN` yields a friendly server-misconfiguration error. **Requires the `GITHUB_TOKEN` env var** (see Configuration).

Home Page (`components/home/*`)

- `HeroSection.tsx` — Main hero banner with video/image
- `WhatsOnSection.tsx` — the What's On block, all inside the `max-w-7xl` container: header + "View Church Calendar", the event carousel (max 6 events, featured one pinned first), and the "View All" button. Falls back to three standing weekly gatherings when the ChurchSuite feed is empty or down.
- `EventsCarousel.tsx` — the horizontal scroll track with prev/next arrows (client). Its `pt-3 pb-10` is load-bearing: `overflow-x` clips the cross axis too, so without it the cards' hover lift and drop shadows are sliced off. The arrows use `top-3 bottom-10 my-auto` rather than `top-1/2` so that asymmetric padding doesn't push them below the card midline, and the track's negative margin tracks the container padding at `lg` (`-mx-8` , not a fixed `-mx-4`).
- `MinistriesGrid.tsx` — Ministry cards (kids, youth, etc.)
- `UpcomingSermons.tsx` — Latest sermons carousel
- `CTAButtons.tsx` — Prominent call-to-action buttons

Shop (`components/shop/*`)

- `ShopProductGrid.tsx` — client wrapper around the `/shop` grid; derives a category filter chip row from the distinct `product.category` values present ("All" + one chip per category, alphabetical), filters the grid client-side on click, and shows an empty state ("Nothing here yet — Try a different category.") when a filter matches nothing. Renders no chips at all if every product shares one/no category.
- `ProductCard.tsx` — storefront product tile (editorial `.shop-card` hover wipe)
- `ProductGallery.tsx` — main image + thumbnail strip (client)
- `ProductBuyPanel.tsx` — colour swatches + size pills + quantity + add-to-basket (client)
- `CartButton.tsx` — header basket icon with live item-count badge (client)
- `CheckoutForm.tsx` — Stripe Express Checkout Element (Apple Pay / Google Pay / Link, shown only when a wallet is available) above the Payment Element card form; both confirm the same PaymentIntent (client)
- `ShopDiagnostics.tsx` — **TEMPORARY.** Mounted for every `/shop` route by `app/shop/layout.tsx` to chase an intermittent blank/white page in Chrome. Renders nothing. See "Shop blank-page diagnostic" below; delete this component, the layout and `/api/shop-diagnostics` once the cause is confirmed.

Shop blank-page diagnostic (temporary)

The bug: `/shop` and its sub-routes intermittently paint blank/white in Chrome. Ruled out by investigation — server errors (12/12 identical 200s), JS heap growth (flat ~15MB), image weight (largest source image 110KB), console errors (none) and a renderer crash (Chrome's Crashpad directory empty). Not reproducible on demand, hence instrumentation rather than a speculative fix.

Two candidate failure modes need different evidence, which is what the payload is shaped around:

Mode	What the report looks like
React/JS died	`dom.mainChildren === 0`, plus a `js-error` / `dom-blank` reason
Compositing failed	DOM completely intact, **no** error — the GPU simply never painted

The second mode is invisible to any automatic check, which is why a **manual hotkey (Ctrl/Cmd + Shift + 9)** exists: a `manual` report showing a healthy DOM and zero errors is itself the finding, because it eliminates the first mode.

Automatic tripwires: `window.onerror`, `unhandledrejection`, a 1x1 WebGL context listening for `webglcontextlost` (the only JS-visible signal of a GPU process reset), a `PerformanceObserver` for long tasks, and a 3s poll for an emptied `<main>`. Observation is deliberately passive — `PerformanceObserver` rather than a `requestAnimationFrame` loop — so the instrument does not perturb the compositing behaviour it is measuring.

`render.refractActive` is the key field: it records whether the SVG refraction `backdrop-filter: url(#glass-refract)` is live at that moment. It differs by route — the `/shop` index disables it via `html:has(.shop-page)` in `globals.css`, the sub-routes do not — so it discriminates the prime suspect automatically.

Per-route state: the report budget, event buffer and timers all reset on `pathname` change. The component lives in the shop layout and survives client-side navigation, so without that reset the budget would be spent once per browsing session and the instrument would fall silent exactly when the bug next appeared.

> **Apple Pay:** enabled automatically by `automatic_payment_methods` — no extra API params. The only prerequisite is a **registered payment-method domain**; Stripe hosts the Apple domain-association file, so there is no `/.well-known` file to deploy. Register the live domain(s) once via Stripe Dashboard → Settings → Payment methods → Apple Pay, or run `scripts/register-apple-pay-domain.mjs` with the live `STRIPE_SECRET_KEY` (e.g. `destinytees.uk` and `www.destinytees.uk`). Apple Pay renders on supported devices/browsers only (Safari on Apple hardware).

API Routes & Server Actions

Server Actions vs. API Routes

Server Actions (`app/*/actions.ts`): - Call from client components directly - Typed, type-safe - Auto-serialize arguments & return values - Good for: Form submission, data mutation - Example: `app/contact/actions.ts`

API Routes (`app/api/*/route.ts`): - HTTP endpoints - Called from fetch() or external integrations - Good for: Webhooks, public APIs, external tools - Example: `/api/chat` (Smart Search)

Key Server Actions

`app/contact/actions.ts`

```
"use server";

export async function submitContactForm(data: ContactFormData) {
  // Validate input
  const parsed = contactFormSchema.parse(data);

  // Rate limit: max 5 submissions per IP per hour
  const remaining = await rateLimit(request.ip, "contact-form", 5, 3600);
  if (remaining <= 0) throw new Error("Too many submissions. Try again later.");

  // Insert into database
  const supabase = createServiceClient();
  await supabase.from("contact_messages").insert({
    name: parsed.name,
    email: parsed.email,
    subject: parsed.subject,
    message: parsed.message
  });

  // Send notification email to admin
  await sendEmail({
    to: "hello@destinytees.uk",
    subject: `New contact form: ${parsed.subject}`,
    html: renderEmailTemplate("contactNotification", parsed)
  });

  return { success: true };
}
```

`app/hire/actions.ts`

Similar structure to contact form: - Validate form data - Rate limit - Insert to `hire_enquiries` table - Send email notification

`app/connect-card/actions.ts`

- Submit prayer request
- Submit connection card (first-time visitor form)
- Inserts to `contact_messages` with category prefix
- Sends email to prayer team

`app/jobs/actions.ts`

```
export async function applyForJob(jobId: string, formData: ApplicationData) {
  // Validate
  const parsed = applicationSchema.parse(formData);

  // Upload CV to storage
  const file = parsed.cv;
  const { path } = await uploadToStorage(file, `job-cvs/${jobId}`);

  // Insert application
  await supabase.from("job_applications").insert({
    job_id: jobId,
    name: parsed.name,
    email: parsed.email,
    phone: parsed.phone,
    cv_url: path,
    cover_letter: parsed.coverLetter
  });

  // Send confirmation email to applicant
  await sendEmail({
    to: parsed.email,
    subject: "We've received your application",
    html: renderEmailTemplate("applicationReceived", { jobTitle })
  });

  return { success: true };
}
```

Admin API Routes

`POST /api/admin/redirects`

```
// Create or update redirect
// Body: { slug, targetUrl, label, active }
// Logic:
// 1. Check auth
// 2. Insert/update redirects table
// 3. Trigger full rebuild (redirects are in next.config.ts)
// 4. Return success or error
```

`GET /api/admin/events`

```
// Event picker feed for /admin/featured-event.
// Projects the ChurchSuite index to { slug, identifier, sequence, id, name,
// start, end, endsAt, image, category, location, sessionCount }.
// force-dynamic — admin must never see a stale list.
```

`GET|PUT /api/admin/featured-event`

```
// Read/write the featured_event singleton: target + hero overrides.
// PUT deletes the superseded image from `popup-images`, then
// revalidatePath("/whats-on") and revalidatePath("/").
// Deliberately never writes popup_* — see /api/admin/featured-event/popup.
```

`PUT /api/admin/featured-event/popup`

```
// Partial update of the popup_* columns only. A full-row upsert from the
// event-popup admin page would blank every hero override.
// Rejects popup_active when no event is featured, with a readable message.
// No revalidatePath — app/layout.tsx reads the popup with noStore().
```

`POST /api/admin/popup/upload`

```
// Upload image for pop-up (also used by the featured-event admin)
// FormData: { file: File, prefix?: "popup" | "featured-event" | "event-popup" | "nfc" }
// `prefix` is allowlisted rather than interpolated so it can't escape into a path.
// Logic:
// 1. Check auth
// 2. Upload to storage bucket
// 3. Return public URL or signed URL
// 4. Admin uses URL in pop-up form
```

`GET|POST|PUT|DELETE /api/admin/nfc`

```
// CRUD for the nfc_tiles rows behind /nfc. Auth is free: middleware.ts matches
// /api/admin/:path*, so an unauthenticated caller never reaches the handler.
//
// Writes go through one buildPayload() validator so POST and PUT can't drift:
//   - mode = "embed" requires an embed_url that passes isEmbeddable()
//     (hostname ends with churchsuite.com). Anything else sets X-Frame-Options
//     and would render as a blank white box, so it's rejected with a sentence
//     telling the admin to use a details tile instead.
//   - mode = "event" takes only an event_identifier from the client and
//     re-resolves it server-side via resolveEvent() → getEventIndex(). That's
//     where embed_url (the signup URL + #form_event_signup), event_slug,
//     event_name, event_sequence and event_ends_at are written. Three named
//     failures, each naming the way out: the event is gone from the calendar,
//     it takes no signups, or it books through a site that refuses framing.
//   - cta_link must start with "/" or "https://".
//   - Length limits mirror the DB checks so the admin gets a readable error
//     rather than a Postgres constraint string.
//
// PUT/DELETE remove the superseded image from `popup-images` only *after* the
// row write succeeds, so a failed write can't orphan the live artwork.
// No revalidatePath — getNfcTiles() reads with noStore().
```

`POST /api/admin/revalidate`

```
// Manually trigger ISR for a path
// Body: { path: string }
// Logic:
// 1. Check auth
// 2. Call revalidatePath(path) or revalidateTag(tag)
// 3. Return success
```

`GET /api/admin/posts/check-slug`

```
// Live slug-availability check for the post editor
// Query: ?slug=<candidate>&excludeId=<postId?>
// Logic:
// 1. Delegate to lib/posts-slug.ts → checkSlug() with the service client
// 2. Always returns 200 with { available, slug, reason? } so the editor
//    can show inline validation feedback while typing (excludeId lets an
//    existing post keep its own slug when editing)
```

Public API Routes

`POST /api/chat`

```
// AI-powered conversational Smart Search (the FloatingSmartSearch widget).
// Body: { messages: { role: "user" | "assistant"; content: string }[] }
// Response: NDJSON stream of { type: "text" | "tool_call" | "tool_result" | "done" | "error", ..

// Logic:
// 1. Requires a valid signed `ts_verified` cookie (Cloudflare Turnstile, see
//    Authentication & Authorization) — 403 if missing/expired
// 2. Rate limit: 20 requests per IP per minute (429 on excess)
// 3. Validate messages: user/assistant roles only (blocks injected system turns),
//    <=2000 chars each, last must be user <=300 chars (else 400)
// 4. getOpenAI() null-check → NDJSON fallback routing to /contact
// 5. Tool-calling loop (up to MAX_TOOL_ROUNDS=4 — enough for search → extract_page
//    → answer, plus headroom) on gpt-4.1-mini:
//    - System prompt: buildSmartSearchPrompt() (lib/siteKnowledge.ts) — church
//      facts + today's date + tool guidance
//    - tools: TOOL_DEFINITIONS (lib/smartSearch/tools.ts) — find_products,
//      get_weather, get_directions, search_web, extract_page
//    - A per-request ToolContext (createToolContext(), tracks seenUrls) is
//      threaded through every executeTool() call so extract_page can only
//      open URLs a search_web call in this same request actually returned
//    - Streams assistant text as `text` events; emits a `tool_call` event per
//      tool invoked (drives the client's "Searching the web…" status line),
//      runs tool calls, emits each result as a `tool_result` event, feeds
//      results back, repeats
// 6. Client (FloatingSmartSearch) accumulates `text` (parsed for PAGE/CTA/OPTION)
//    and renders each `tool_result` as a card (ResultCards.tsx)
```

`POST /api/turnstile/verify`

```
// Verifies a Cloudflare Turnstile token (siteverify) and, on success, sets a
// signed `ts_verified` cookie (HMAC via TURNSTILE_COOKIE_SECRET, TTL 30 min —
// lib/turnstile.ts TURNSTILE_SESSION_TTL_MS). Called by both the /login form
// and FloatingSmartSearch before it will hit /api/chat.
// Body: { token: string }  →  200 { success: true } | 403 { success: false }
// Cookie is readable client-side (httpOnly: false) so the widget can check
// hasVerifiedCookie() before re-challenging, but can't be forged — the value
// is `${timestamp}.${hmac}` checked with a timing-safe comparison.
```

`GET /api/events/[slug]/ics`

```
// Calendar download for an event series — the .ics link on every /whats-on/[slug]
// page and event card. Public and outside the /api/admin/* matcher: it serves only
// what the ChurchSuite feed already publishes, so it needs no auth.
// Logic:
// 1. getEventIndex() (lib/events.server.ts), then resolve `slug` via bySlug with the
//    same identifier fallback as the event page (a `-<identifier>` suffix on a
//    renamed/disambiguated URL still downloads instead of 404ing). 404 if unmatched.
// 2. buildIcs({ series, occurrence, baseUrl }) from @destiny/shared builds the VCALENDAR.
//    Without ?occurrence= the file contains every upcoming session, so subscribing to a
//    recurring event adds the whole run at once; ?occurrence=<id> narrows it to one.
// 3. Returns text/calendar as an attachment (filename `<slug>.ics`).
// revalidate = 300; Cache-Control public, s-maxage=300, stale-while-revalidate=600.
```

`GET /api/youtube/videos`

```
// Returns getAllVideos(50) — the sermon archive's video list (lib/youtube.ts).
// Fetched client/server-side by the /sermons page; no separate sync job or
// cache table — YouTube is queried directly on each call.
```

`GET /api/youtube/status`

```
// { quotaExceeded: boolean } — lib/youtube.ts isYouTubeQuotaExceeded().
// revalidate = 3600. Lets the client show a friendly fallback if the
// YouTube Data API daily quota has been used up.
```

`GET /api/youtube/thumbnail/[id]`

```
// Proxies a YouTube video thumbnail image (avoids hot-linking i.ytimg.com directly).
```

`GET /api/youtube/live`

```
// Livestream status, polled client-side every 30s by LiveContext.
// Response: { live, videoId, title?, startedAt?, scheduledFor?, checkedAt }
// dynamic = "force-dynamic"; Cache-Control: s-maxage=30, stale-while-revalidate=30
//
// ?debug=1 additionally returns `source` — which detection layer answered
// (channel-page | videos.list | confirmed-offline | no-signal | disabled |
// no-channel | error) — and sets no-store. That is the fastest way to work out
// why the banner is or isn't showing in production without a redeploy.

// Backed by lib/youtube.ts getLiveStatus() — see Libraries & Utilities.
```

> There is no `/api/webhooks/vercel` or GitHub webhook route, and no `youtube-sync` cron/cache job — `app/api/webhooks/` currently only contains `stripe/` (see Shop API below). YouTube data is fetched live on each request, not synced to a cache table.

`GET /api/health/smart-search` — self-healing health check (cron)

```
// Vercel Cron endpoint that keeps Smart Search from showing a broken chat when
// OpenAI is unreachable. Auth: Authorization: Bearer <CRON_SECRET> (Vercel Cron)
// or a manual ?secret=<CRON_SECRET> run; if CRON_SECRET is unset (local/dev) it
// runs open. maxDuration = 30.
//
//   1. Reads service_status via getSmartSearchStatus(). While healthy it only
//      actually pings once next_check_at is due (daily), so an hourly cron can
//      retry fast while DOWN but stays cheap while UP ({ skipped: true } otherwise).
//   2. Live-pings OpenAI (SMART_SEARCH_MODEL, max_tokens 1, 15s timeout, no retry).
//   3. On success  → enabled = true, next check in 24h, failures reset to 0.
//      On failure   → enabled = false, next check in 1h, failures++, reason stored.
//   4. Emails an alert (lib/smartSearchAlertEmail.ts, via Resend) ONLY on a state
//      change — healthy→down or down→recovered — so the inbox never gets hourly spam.
//
// app/layout.tsx reads isSmartSearchEnabled() to decide whether to mount
// FloatingSmartSearch, so a disabled service simply hides the feature site-wide.
```

Training API

```
// PUBLIC (outside the middleware matcher)
POST /api/training/unlock
//   Body: { subgroupId, password }. Verifies a sub-group password against the
//   hashed password_hash (server-side only; the hash is never sent to clients).
//   On success sets a scoped cookie so the gated sub-group's posts render.

POST /api/training/posts/[id]/timer
//   Enforces a training post's min_read_seconds "minimum read time" before the
//   member can mark it complete. Stateless — no DB writes; state rides an
//   HMAC-signed, base64url token so it survives across serverless instances.
//     action: "start"  → returns { token } signed over { postId, startedAt }.
//     action: "verify" → re-checks the token (timingSafeEqual) and postId, then
//                        compares elapsed time to minReadSeconds. Returns
//                        { success:true } once met, else 403 with { remaining }.
//   HMAC key precedence: TRAINING_TIMER_SECRET → HMAC of SUPABASE_SERVICE_ROLE_KEY
//   (never used raw) → a per-boot random key (last resort, instance-local).
//   Driven by components/training/CompleteButton.tsx on the training post page.
```

Shop API

```
// PUBLIC (outside the middleware matcher)
POST /api/store/checkout
//   Body: { items: [{ variantId, quantity }], customer: { name, email, phone?, notes? } }
//   Recomputes every price + stock from the DB (client prices never trusted),
//   creates a pending order + items, creates a Stripe PaymentIntent, returns
//   { clientSecret, orderNumber }. Rate-limited per IP.

POST /api/webhooks/stripe
//   Stripe signature-verified (STRIPE_WEBHOOK_SECRET), runtime = nodejs, raw body.
//   payment_intent.succeeded → order 'paid', decrement variant stock, Resend
//   confirmation emails (customer + church). Idempotent. payment_failed → 'cancelled'.

POST /api/store/checkout/bypass
//   TEST ONLY. 404 unless server env SHOP_TEST_BYPASS=1. Creates a real order and
//   finalises it WITHOUT Stripe (paid, stock decremented, emails) so the full flow
//   can be demoed without a payment. The checkout page shows a "Complete test order"
//   button when NEXT_PUBLIC_SHOP_TEST_BYPASS=1. Never set either flag in production.

// Shared order logic (pricing recompute, order creation, paid-finalisation) lives
// in lib/checkout.server.ts and is used by checkout, the webhook, and the bypass.

POST /api/shop-diagnostics
//   TEMPORARY — remove with components/shop/ShopDiagnostics.tsx once the /shop
//   blank-page bug is identified. Receives a client snapshot captured when /shop
//   renders blank and does two things with it:
//     1. console.error → Vercel runtime logs. Always runs; the reliable channel.
//     2. Files/comments a GitHub issue via the GITHUB_TOKEN already used by
//        components/report-bug/actions.ts, deduped by reason+route (label
//        "shop-blank-page") so a recurring bug appends to one issue.
//   Public and unauthenticated but fenced: same-origin only (403), 32KB body cap
//   (413), 5 reports per IP per 10 min (throttled), and localhost skips the
//   GitHub write so dev noise never reaches the issue tracker.

// ADMIN (gated by middleware, site_editor role)
GET|POST            /api/admin/store/products
GET|PUT|DELETE      /api/admin/store/products/[id]        // PUT reconciles variants
POST|DELETE         /api/admin/store/products/[id]/images // sharp → WebP → product-images bucket
GET                 /api/admin/store/orders
GET|PATCH           /api/admin/store/orders/[id]          // PATCH updates fulfilment status
GET|POST            /api/admin/shop-hero                  // list / create hero slides
PATCH|DELETE        /api/admin/shop-hero/[id]             // update (content/active/sort_order) /
POST                /api/admin/shop-hero/upload           // sharp → WebP → shop-hero-images buck
```

Content Blocks

Admin-authored, configurable components that staff drop into a page from the **Blocks** sidebar in the editor — FAQ accordions, callouts, quotes, card grids, image galleries and button rows. The point is that a new FAQ or a new set of cards no longer needs a developer and a deploy.

Why it is built this way

A JSONB page builder (`builder_pages.document` , `studio_components`) was built here previously and removed in `supabase/migrations/20260711_06_remove_page_builder.sql` . This deliberately does **not** rebuild that.

Content stays exactly what it always was: **a single HTML string** in `posts.body` (and `jobs.description` , `training_posts.body` , `products.description`). There is no migration, no new table, and no second representation of a page that can drift out of sync with the first. A block is just a marker inside that string.

Wire format

A block serialises into the stored HTML as one **empty, top-level** div:

```
<div data-block="faq" data-block-version="1" data-props="{&quot;heading&quot;:&quot;FAQs&quot;,…
```

All of the block's settings are JSON in `data-props` . `components/content/ RichContent.tsx` splits the stored HTML on these markers and renders the real component in their place.

That split is done with a regex rather than an HTML parser, which is only safe because of **two invariants**. Both are enforced, not assumed:

1. **The attribute value never contains `"` , `<` or `>` .** The browser's DOM serialiser escapes `&` , `"` and U+00A0 in attribute values — but *not* `<` and `>` . So `encodeProps` escapes those itself (plus U+2028/9), with a dev-mode invariant throw. The value provably matches `/^[^"<>]*$/` .
2. **A block can never nest.** Blocks belong to the ProseMirror group `dcblock` , and `Document` is extended to `content: "(block | dcblock)+"` . Since `blockquote` and `listItem` declare `content: "block+"` , a block inside one is structurally impossible — so a block div is always a self-contained top-level token and can never split surrounding markup into unbalanced halves.

`tests/unit/blocks-serialize.spec.ts` and `blocks-wire-format.spec.ts` guard both, and `RichContent` warns in development if the format ever drifts.

Why not `html-react-parser`

Five live routes render stored HTML. A parser would re-parse and *re-serialise* all of that existing content as React elements — `class` → `className` , style strings to objects, boolean attributes, whitespace — and every one of those is a chance to change output on pages nobody intended to touch. The split approach passes block-free content through the identical code path it used before, so adopting it everywhere was a provable no-op. It also avoids shipping ~15–20KB of parser to every public page.

The three layers

```
components/blocks/          SHARED — no "use client"
components/content/         RichContent — the public renderer
components/admin/blocks/    "use client" — TipTap nodes, sidebar, inspector
```

Block components carry no `"use client"` directive. That makes them RSC "shared" components: they server-render on the public page with **zero client JS**, and the identical module renders inside the editor's React node view. That is what makes the editor genuinely WYSIWYG rather than a placeholder preview — and it is why no block may import `AnimateIn` , `motion` , `next/image` or anything else client-only.

Adding a block

1. Create `components/blocks/<name>/{def.ts,<Name>Block.tsx}`
2. Import the def in `components/blocks/registry.ts` and add it to `BLOCK_LIST`

That is the whole checklist. One `BlockDefinition` drives the sidebar tile, the settings form, the TipTap node and the public renderer. **If adding a block ever requires touching the sidebar, the inspector, the TipTap plumbing or RichContent, the abstraction has leaked — fix that rather than special-casing.**

Things that will bite you:

- **Every schema key must have `.default()`.** Props are parsed as `{ ...defaults, ...stored }`, which is what lets content authored before a field existed keep working. Without the default, `safeParse` fails and the block silently resets to defaults, losing the author's content.
- **A `<select>` always yields a string.** A schema expecting `z.literal(2)` will fail to parse and reset the block. Model numeric choices as string enums (`z.enum(["2","3"])`).
- **Never rename a `name`.** It is the wire format, written into every page using the block.
- **Anything used as `href` or `src` must go through `safeUrl`.** React escapes props it renders as children but not values placed in those attributes.
- **Blocks render in four different column widths** (posts, training, jobs, shop). Size with `@container` queries and `cqi` units, never viewport breakpoints. `[data-dc-block]` carries `container-type: inline-size`.
- Block widths are `column` or `wide` only. There is deliberately no full-bleed: at ≥1600px `app/[slug]/page.tsx` places the article in an off-centre grid track beside the promo rails, so the usual `margin-inline: calc(50% - 50vw)` trick would be wrong there.

Key files

File	Role
`components/blocks/types.ts`	`BlockDefinition`, `FieldSchema`
`components/blocks/serialize.ts`	`encodeProps` / `decodeProps` / `unescapeAttr` / `BLOCK_RE`
`components/blocks/tokens.ts`	Shared design vocabulary — card shell, eyebrow, heading scale, tones, buttons, `safeUrl`
`components/blocks/registry.ts`	`BLOCK_LIST` — the single source of truth
`components/content/RichContent.tsx`	Public renderer + merged JSON-LD
`components/admin/blocks/createBlockNode.tsx`	Generic TipTap node factory
`components/admin/blocks/UnknownBlockNode.tsx`	Data-preserving fallback
`components/admin/blocks/BlockNodeView.tsx`	Editor chrome, drag grip, click shield
`components/admin/blocks/BlockPalette.tsx`	The block inserter — sidebar on desktop, searchable grid in the sheet
`components/admin/blocks/BlockInspector.tsx`	Schema-driven settings panel
`components/admin/blocks/BlockOutline.tsx`	Jump list of every block in the document
`components/admin/blocks/blockCommands.ts`	Move / duplicate / delete / add-paragraph, by position
`components/admin/blocks/BlockTools.tsx`	Sheets + the mobile selected-block toolbar
`components/admin/Sheet.tsx`	The admin bottom sheet (grabber, detents, drag-to-dismiss)
`app/dev/blocks`	Development-only gallery; 404s in production

Mobile is a different interaction, not a narrower one

Desktop edits a block by hovering it, reading the chrome bar that appears and clicking a 24px control. None of those three steps exist on a touch screen, so below `lg` the blocks UI is replaced rather than reflowed — the same model every touch editor converged on:

- **Tap a block to select it**, and a toolbar docks to the bottom of the screen with that block's actions at 44px: move up, move down, its name, Settings, and a "more" action sheet holding duplicate / add-paragraph / delete. The hover-revealed chrome bar in `BlockNodeView` is `hidden lg:flex`; what mobile keeps is a standing label chip above each block, because with no hover there is otherwise nothing saying a block is one tappable object.
- **Settings open at the `half` detent**, so the block stays on screen above the sheet and visibly updates as you type. That live feedback is most of what makes the settings legible on a small screen.
- **With nothing selected**, the settings sheet lists the blocks in the page (`BlockOutline`) instead of saying "click a block" — the canvas is behind the sheet, so that was advice a thumb could not follow.
- **The inserter is a searchable two-column grid** in the sheet, and its copy does not mention dragging, which touch cannot do.

- **Every inspector input is 16px on touch** (`fieldInputClass`). Below that, iOS Safari zooms the page on focus and does not zoom back out, which put the block being edited off-screen on the first tap of the first field.
- Repeater rows keep move up/down in the header and moved duplicate/remove into the open row as labelled buttons — four 24px icon buttons in a row put "remove" a thumb-width from "duplicate".
- `RichTextEditor` 's formatting toolbar is one horizontally scrolling row below `lg` . Wrapped, its nineteen controls took six rows — about a third of a phone screen — above an editor with no room left.

`components/admin/Sheet.tsx` is the shared surface underneath all of it: grabber, drag down to dismiss (full → half → closed) and up to expand, `auto` / `half` / `full` detents, safe-area inset, body-scroll lock, and it lifts above the keyboard via `useKeyboardInset` . Escape is handled in the **capture** phase: these sheets open on top of admin modals that close on Escape too, and the modal's document listener is registered first, so nothing in the bubble phase can stop it.

It portals to `document.body` at `z-[110]` (the toolbar at `z-[105]`). Portalled because the admin modal backdrops use `backdrop-filter` , which makes them a containing block for `fixed` descendants — the toolbar would anchor to the modal instead of the screen. The z-indexes sit above the admin modals (`z-50`) and the dev sandbox (`z-[100]`), and below `DialogProvider` (`z-[200]`) so the editor's link prompt still opens over a sheet rather than underneath one.

Embeds go through the cookie-consent gate

`MediaEmbed` and `ChurchSuiteEmbed` hold third-party iframes behind a consent placeholder until the visitor opts in to media cookies (`consent.media`). The Video and ChurchSuite form blocks wrap those components rather than emitting their own `<iframe>` , so admin-authored embeds behave like the rest of the site.

The old toolbar buttons did not: they wrote a bare `<iframe>` into the stored HTML, which loads YouTube or ChurchSuite — and their cookies — before the visitor has agreed to anything. **Content authored with those buttons still renders that way**; this only changes what new content does. Worth a pass over existing posts if that matters.

Two consequences:

- These two blocks are the only ones that ship client JavaScript, because the consent hook needs it. Everything else stays a shared component and renders with zero JS.
- The Custom embed block *can't* be gated — we have no idea what's in it. It sits in the sidebar's Advanced group precisely so it's the last thing reached for, and its help text points at the other two.

Non-obvious things that will waste your time

- **`UnknownBlockNode` is not optional.** ProseMirror drops elements its schema doesn't recognise. If a block is removed from the registry, or a deploy is rolled back, opening an affected page would quietly discard the block and the author's next save would write that loss to the database permanently. This node round-trips the marker verbatim (it does **not** re-encode `data-props` , since there's no schema to re-encode against) and shows a placeholder.
- **The drag grip must not be a `<button>` .** TipTap's `NodeView.stopEvent` returns early for `INPUT/BUTTON/SELECT/TEXTAREA` targets *before* the bookkeeping that records a drag started from `[data-drag-handle]` . A `<button data-drag-handle>` is silently undraggable. It is a `<span role="button">` . The other chrome buttons *are* real buttons, because that same early return is what they want.

- **Never pass a custom `stopEvent` to `ReactNodeViewRenderer`** — it short-circuits the whole default, including that drag bookkeeping.
- **`renderHTML` is hand-written, not `mergeAttributes`.** That pins attribute order so `BLOCK_RE` cannot drift. "Tidying" it back would make every block silently vanish from every public page.
- **Do not add `value` to the `[editor]` effect in `RichTextEditor.tsx`.** The parent re-renders with a new `value` on every keystroke, so a `value` dep is an infinite loop — and even a guarded version calls `setContent`, which rebuilds the document, destroys every node view, drops the selection and wipes undo history mid-edit.
- **Blocks are never in the rich-text toolbar.** That toolbar formats the current text selection; blocks are page structure. On desktop the post editor gives them a persistent sidebar and the modal editors get `BlockTools` sheets; on mobile they are sheets plus a docked toolbar for the selected block. All three are outside the formatting strip.
- The inspector debounces writes ~200ms and derives its draft during render rather than syncing via an effect, so a typed word is one undo step and an external Cmd+Z is reflected.

Testing

`npm run test:unit` (or `npx playwright test --project=unit`) runs the unit specs in plain Node — no browser, no dev server, no second test runner. They cover the block wire format against an adversarial corpus (script tags, quotes, HTML entities, U+2028/9, angle brackets, 200-item arrays), block registry integrity (every field name exists in `defaults`; every schema parses `{}`), attribute-order drift, sermon/podcast pairing, course-registry integrity (`course-events.spec.ts` — every type has complete metadata, no type is owned by two admin pages or none), livestream page parsing (`live-detection.spec.ts` — a live broadcast is detected even when recommendation shelves carry `isLiveNow: false` first, a finished broadcast isn't, and a channel-home redirect is a definitive offline rather than an escalation), and Sunday service times across both DST boundaries (`service-times.spec.ts`).

Browser specs run with `npm run test:e2e`. Those needing an admin session (`admin-blocks`, `admin-courses`) skip themselves unless `ADMIN_EMAIL` and `ADMIN_PASSWORD` are set; credentials come from the environment only and live in the gitignored `CLAUDE.local.md`.

Libraries & Utilities

Admin helpers (`lib/adminUpload.ts`, `lib/useIsDesktop.ts`, `lib/useKeyboardInset.ts`, `lib/useIsClient.ts`)

- `lib/adminUpload.ts` — `uploadPostImage(file)` posts to `/api/admin/posts/upload` (auto-rotate from EXIF, resize to max 1600px, re-encode to WebP q82, `post-media` bucket). Extracted from `RichTextEditor`'s inline toolbar handler so the block inspector's image field shares one implementation. Exports `UPLOAD_ACCEPT` and `UPLOAD_MAX_BYTES`, which mirror the route's own limits so an oversized file fails before the upload rather than after.
- `lib/useIsDesktop.ts` — the ≥1024px breakpoint, read **synchronously** via `useSyncExternalStore`. The admin editors branch their whole layout on this, and the obvious `useState(false)` + effect version mounted the mobile tree first and then swapped to a different one, silently destroying and recreating the TipTap instance (and its undo history) on every open. `getServerSnapshot` returns `false` so SSR and the first client paint agree.

- `lib/useKeyboardInset.ts` — how many pixels the on-screen keyboard covers at the bottom of the window, from `visualViewport`. `position: fixed` resolves against the *layout* viewport, which iOS Safari does not shrink for the keyboard, so bottom-anchored sheets and toolbars would otherwise sit underneath it. Thresholded at 120px and rounded: URL-bar collapse and pinch-zoom also move the visual viewport, and an unrounded value hands `useSyncExternalStore` a new snapshot on every scroll frame.
- `lib/useIsClient.ts` — false on the server and the hydrating render, true after. Guards `createPortal(…, document.body)`. `useSyncExternalStore` rather than `useState` + effect because the latter is a synchronous setState inside an effect, which the React lint rules reject as a cascading render.

`lib/welcomeChat.ts`

The scripted conversation behind `FloatingSmartSearch`'s `welcome` mode (see Components). **Church copy and routes are edited here** — the component only renders whatever this file says.

A `ChatNode` is `{ id, message, choices }`. A `ChatChoice` either advances the thread (`next`) or opens a page (`href`), **never both**, so there is never a hidden branch behind a link. Routes are validated against `ALLOWED_PAGES` from `lib/siteKnowledge.ts` — the same allowlist Smart Search uses to reject hallucinated links — with `routeOf()` stripping any `#section` first.

None of that is enforced by types, because a typo'd `next` renders a dead button rather than failing to compile. It is enforced by `tests/unit/welcome-chat.spec.ts`, which checks ids are unique, every `next` resolves, every `href` is a real page, every node is reachable from the root, and every node can eventually reach a page (no dead ends).

`lib/embedLoading.ts`

The stage timings behind `ui/EmbedLoadingOverlay` (see Components), kept apart from the component so the boundaries are plain data and can be unit-tested without a browser — `tests/unit/embed-loading.spec.ts`.

`stageIndexAt(elapsed)` picks the line to show, and `msUntilNextStage(elapsed)` says how long until the next one. Both derive from elapsed milliseconds rather than counting ticks, because browsers clamp timers in a backgrounded tab: a counter would walk through every stage it slept through, one per wake-up, whereas this lands straight on the right line. `msUntilNextStage` returns `null` on the last stage, which is how the component knows to stop scheduling.

`lib/serviceTimes.ts`

Sunday service times in the church's own timezone. The site is served from UTC infrastructure and read on phones set to anything, but "11am" always means 11am in Stockton-on-Tees — so this resolves London wall-clock times to real instants rather than trusting the runtime's local zone. `Intl` only: no date library, and no hardcoded BST/GMT switchover dates to go stale.

- `nextSundayService(from?)` — the start of the next Sunday 11am service. Returns *today's* service right up to the 12:30 finish, so someone watching the stream isn't told the next one is a week away. Steps forward from London noon rather than from `from`, because adding raw 24h multiples across a DST boundary can shunt the result onto the Saturday.

- `formatServiceDay(date)` — "Sunday 14 September" in London's calendar.
- `formatCountdown(target, from?)` — "2 days 4 hours" → "18 minutes"; null once the target has passed.

Used by `components/live/NextServiceCountdown.tsx`. Covered by `tests/unit/service-times.spec.ts`, which pins both DST boundaries and the mid-service behaviour.

Shop (`lib/shop*.ts` , `lib/stripe.ts` , `lib/cart-store.ts`)

- `lib/shop.ts` — client-safe types (`Product` , `ProductVariant` , `Order` , `CartItem`), `formatPrice(pennies)` , `variantPrice` , `fromPrice` , `totalStock` . Prices are integer pennies (GBP).
- `lib/shop.server.ts` (`server-only`) — public read fetchers: `getPublishedProducts()` , `getProductBySlug()` , `getAllProductsAdmin()` (via `getSupabaseAdmin()`).
- `lib/stripe.ts` (`server-only`) — `getStripe()` singleton from `STRIPE_SECRET_KEY` .
- `lib/cart-store.ts` — `useCart` zustand store persisted to `localStorage` (`destiny-cart`), plus `cartCount` / `cartSubtotal` helpers.

`lib/youtube.ts`

`CHANNEL_HANDLE` / `CHANNEL_VANITY` / `CHANNEL_URL` — the church's YouTube channel, in one place. The channel answers on **two different names** and they are not interchangeable:

Form	Value	URL
`@` handle	`DestinyOnlineChurch`	`youtube.com/@DestinyOnlineChurch`
Custom URL	`destinychurchteesvalley`	`youtube.com/destinychurchteesvalley`

`CHANNEL_URL` is the custom-URL form and is what every public link uses (sermons platform chips, `WatchOnYouTubeBand` , `LatestSermonSection` , the org schema's `sameAs`). Mixing the two produced `@DestinyChurchTeesValley` — neither name, and a dead link — which is what the org schema and the `/help` FAQ both used to point at.

Deliberately plain constants rather than env reads: `lib/youtube.ts` is reachable from client components, and a non- `NEXT_PUBLIC_` variable is `undefined` in the browser bundle, which would hydrate a different `href` than the server rendered. Live detection alone may override them server-side via `YOUTUBE_CHANNEL_HANDLE` / `YOUTUBE_CHANNEL_VANITY` .

```
// Wrapper around YouTube Data API v3
export async function getAllVideos(maxResults = 50): Promise<YTVideo[]> {
  const youtube = google.youtube({
    version: "v3",
    auth: process.env.YOUTUBE_API_KEY
  });

  // Search for videos in our channel
  const { data } = await youtube.search.list({
    part: "snippet",
    channelId: process.env.YOUTUBE_CHANNEL_ID,
    maxResults,
    order: "date",     // Newest first
    type: "video"
  });

  // Transform to YTVideo type with metadata
  return data.items.map((item) => ({
    id: item.id.videoId,
    title: item.snippet.title,
    description: item.snippet.description,
    thumbnail: item.snippet.thumbnails.high.url,
    publishedAt: item.snippet.publishedAt,
    // ... extract speaker, series from description/title
  }));
}

export async function getLatestVideo(): Promise<YTVideo | null> {
  const videos = await getAllVideos(1);
  return videos[0] ?? null;
}

// Livestream detection — used by the /live page and the "WE ARE LIVE" banner.
// Zero-quota on the happy path; escalates only when scraping is inconclusive.
export async function getLiveStatus(): Promise<LiveStatus> { /* ... */ }
```

How `getLiveStatus()` decides, in order:

1. **Scrape the `/live` URL**, trying all three forms of the same channel in order: `youtube.com/channel/{id}/live`, `youtube.com/@DestinyOnlineChurch/live`, `youtube.com/destinychurchteesvalley/live`. The handle and custom URL are the safety net for a missing or stale `YOUTUBE_CHANNEL_ID`, which previously made detection fail silently and permanently. A definitive verdict from any one form breaks the loop — the later forms are only tried when the earlier one came back blocked or unreadable, so the offline path stays at exactly one fetch.
2. **Parse properly.** `parseLivePage()` brace-matches the `ytInitialPlayerResponse` blob out of the HTML and reads `videoDetails.isLive` / `liveBroadcastDetails.isLiveNow` from *that object*. The old code regex-matched `"isLiveNow":(true|false)` anywhere in the page — and YouTube emits `ytInitialData`, full of recommendation shelves each carrying their own `isLiveNow`, **before** the player response. The first match was routinely an unrelated video's flag, so real broadcasts were scored offline. It also never distinguished `isLiveContent` ("was a broadcast at some point", true of every past Sunday) from `isLive` ("streaming right now").
3. **A clean "no" costs nothing.** A watch page with an `endTimestamp`, or a redirect to the channel home, returns `offline` outright — that is the path taken every minute of every day the church isn't streaming, and it must never spend quota. All four channel-home canonical forms (`/channel/UC…`, `/@handle`,

`/c/name`, bare custom URL) are recognised; treating any of them as unreadable would escalate to an API call on every check.

4. **Inconclusive results escalate** — consent wall, bot check, or an unrecognised shell. First the free second opinion: `embed/live_stream?channel={id}`, a few KB rather than a few MB and not consent-gated. Then the RSS feed's recent uploads (also free). Then **one** `videos.list` call over the collected candidate ids (1 unit) for YouTube's own `liveBroadcastContent === "live"` verdict.
5. **Never `search?eventType=live`** — 100 units a call would burn the entire 10,000/day quota inside an hour of polling, taking the sermon pages down with it.

Caching. The scrape uses `cache: "no-store"` and `getLiveStatus()` memoises the result in-process instead (30s while live, 60s otherwise, with concurrent callers sharing one detection pass). A watch page is several MB — past the 2MB ceiling on Next's fetch data cache — so the previous `next: { revalidate: 60 }` was caching nothing and re-scraping YouTube on every render of every page, which is both slow and enough traffic to earn a bot check. On a detection failure the last known answer is kept rather than yanking a running stream off the page.

Fails closed. Any unexpected state, or `LIVE_DISABLED=1`, returns `{ live: false }`.

Why Supabase Storage isn't used for videos: - YouTube handles CDN/caching automatically - Bandwidth costs for video are lower on YouTube - YouTube playlist/channel management features - Comments, engagement metrics built-in

---

`lib/smartSearch.ts`

Parses the LLM response and validates navigation:

```
export interface SmartSearchResult {
  answer: string;              // Main prose answer
  page: string | null;         // Valid page URL (or null)
  ctaLabel: string | null;     // Button text
  options?: string[];          // For clarifying questions
}

export function parseAnswer(raw: string): SmartSearchResult {
  // Extract OPTION: lines (clarifying question choices)
  const options = Array.from(
    raw.matchAll(/^\s*(?:[-*•]\s*)?OPTION:\s*(.+?)\s*$/gim)
  )
    .map((m) => m[1])
    .slice(0, 4);  // Max 4 options

  // Strip OPTION/PAGE/CTA lines from prose
  const prose = raw
    .replace(/^\s*OPTION:\s*.+$/gim, "")
    .replace(/^\s*PAGE:\s*\S+/gim, "")
    .replace(/^\s*CTA:\s*.+$/gim, "")
    .trim();

  // If options present, this is a clarifying question
  if (options.length >= 2) {
    return { answer: prose, page: null, ctaLabel: null, options };
  }

  // Extract PAGE and CTA from end of response
  const pageMatch = raw.match(/^\s*PAGE:\s*(\S+)\s*$/im);
  const ctaMatch = raw.match(/^\s*CTA:\s*(.+?)\s*$/im);

  let page: string | null = null;
  if (pageMatch) {
    const candidate = pageMatch[1].replace(/[.,)"']+$/, "");
    // Validate against allowlist (prevents hallucinated links)
    if (ALLOWED_PAGES.has(candidate)) page = candidate;
  }

  let ctaLabel: string | null = null;
  if (page && ctaMatch) {
    ctaLabel = ctaMatch[1].trim();
  }

  return { answer: prose, page, ctaLabel };
}

export const FALLBACK_ANSWERS = {
  tooShort: "Ask me anything about Destiny...",
  unavailable: "I can't reach the assistant right now...",
  empty: "I'm not totally sure on that one..."
};

export function cooldownAnswer(): SmartSearchResult {
  // User hit rate limit
  return {
    answer: "You've made a lot of searches in a short time...",
    page: "/contact",
    ctaLabel: "Contact Us"
```

```
  };
}
```

Key Security:

1. **Allowlist validation** — Page URLs must be in `PAGE_INTENTS`; any hallucinated links are dropped
2. **Rate limiting** — Max 5 searches per IP per minute
3. **Fallback answers** — If service unavailable, still show a friendly response (never "error")

`lib/siteKnowledge.ts`

Single source of truth for the AI assistant. Its system prompt also includes:

- **Charity/company identity guardrail:** the church's registered charity number (1119951) and company number (06261423), plus an explicit warning that other unrelated organisations share the "Destiny Church" name (notably a Scottish charity, SC017898) — the model is told to confirm any record it reads matches before quoting a figure from it.
- **Widened, tool-first web search guidance:** the model is told to prefer calling `search_web` / `extract_page` over guessing or deflecting for real-world or financial questions about the church (UK charities publish their accounts), reserving the `/contact` fallback for genuinely unpublished internals (staff pay, member data) or requests with no connection to Destiny at all.

```
export const PAGE_INTENTS = [
  { href: "/give", cta: "Give Now", intent: "giving, donations, bank details..." },
  { href: "/visit", cta: "Plan Your Visit", intent: "visiting, first time..." },
  { href: "/sermons", cta: "Watch Sermons", intent: "sermons, messages..." },
  { href: "/live", cta: "Watch Live", intent: "livestream, live service, Sundays at 11am" },
  { href: "/shop", cta: "Browse Merch", intent: "shop, apparel, merch, buy" },
  { href: "/help", cta: "Help Centre", intent: "help, FAQ, questions" },
  // ... 17 more pages (kids, youth, Alpha, serve, connect, missions, etc.)
];

export const ALLOWED_PAGES = new Set(PAGE_INTENTS.map(p => p.href));

// System prompt sent to OpenAI (truncated here):
const SEARCH_INSTRUCTIONS = `
You are a friendly assistant for Destiny Church Tees Valley.

HOW TO REPLY:
- Write 2–5 conversational sentences
- Never restart the question or use full church name
- Append PAGE: and CTA: tags ONLY if relevant:
  PAGE: /the-path
  CTA: Short Action Label

GROUNDING (MOST IMPORTANT):
- Only state facts in the KNOWLEDGE section below
- Do NOT guess, invent, or assign roles unless listed
- If you don't know, warmly say so and point to /contact

CLARIFYING QUESTIONS:
- If a request is ambiguous, ask ONE short question with 2–4 OPTION: choices
- Do NOT ask follow-ups for simple factual questions

KNOWLEDGE:
... (church-specific facts: pastor names, service times, livestream, shop, mission, etc.) ...
`;
```

Why this pattern?

1. **Prevents hallucination** — Model is grounded in actual church knowledge
2. **Limits links** — Only allows pages we've defined
3. **Consistent tone** — Friendly, warm, not corporate
4. **Scalable** — Easy to update facts without retraining

`lib/nfcTiles.ts` / `lib/nfcTiles.server.ts`

The tile list behind `/nfc`, deliberately split in two.

```
// lib/nfcTiles.ts — client-safe: types, fixtures, pure helpers
export type NfcTileMode = "embed" | "info" | "event";
export const PINNED_TILES: NfcTile[];               // Connect Card, Giving — always first
export const SIGNUP_ANCHOR: string;                 // "#form_event_signup"
export const NFC_TILE_COLUMNS: string;              // the explicit select list
export function isEmbeddable(url: string): boolean; // hostname ends with churchsuite.com

// lib/nfcTiles.server.ts — `import "server-only"`
export async function getNfcTiles(): Promise<NfcTile[]>;
```

Why the split: `/admin/nfc` is a client component and imports `PINNED_TILES`. Event resolution needs `getEventIndex()` from `lib/events.server.ts`, which is `server-only` — putting the two in one file would break the admin bundle. Keeping them apart also stops the service client being reachable from a client bundle at all.

`getNfcTiles()` is `noStore()` + service client, returns `[...PINNED_TILES, ...activeRows]`, and **catches everything** — a Supabase outage returns the two fixtures rather than a blank page, which matters because the page's whole audience is holding a phone in a service right now.

`mapRow()` re-checks `isEmbeddable()` on read, not just on write: a row that somehow holds a non-framable URL is demoted to `info` mode so the tile shows copy and a link instead of a dead white iframe. `isEmbeddable` is the same rule as `components/events/EventSignupButton.tsx` — only our own ChurchSuite subdomain permits framing.

Event tiles cost a feed fetch only when a row actually has `mode = "event"`. For each one, `resolveEventTile()`: - drops the tile when `event_ends_at` has passed (a *server-side* clock read — the same reason `EventsGrid` and `EventCard` are clock-free is why this can't move to the client); - looks the series up by identifier, then by `event_sequence` — ChurchSuite reissues occurrence identifiers when a series is edited, so the sequence is the more durable key; - on a hit, refreshes the signup URL, the slug and `event_ends_at` from the feed, and fills a blank subtitle with `formatKicker()` so the tile face carries the live date; - on a miss — feed down (`fetchChurchSuiteEvents` returns `[]` on any error) or the event pulled from ChurchSuite — keeps the stored snapshots, which is why the signup URL is snapshotted into `embed_url` at write time.

`lib/pageContent.ts`

Fetch dynamic content from the `page_content` table:

```
export async function getPageContent(page: string, key: string): Promise<string> {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("page_content")
    .select("value")
    .eq("page", page)
    .eq("key", key)
    .maybeSingle();
  return data?.value ?? "";
}

// Used by:
export async function getGivingDetails() {
  return Promise.all([
    getPageContent("give", "account_name"),
    getPageContent("give", "sort_code"),
    getPageContent("give", "account_number"),
    getPageContent("give", "text_keyword"),
    getPageContent("give", "text_number")
  ]);
}

export async function getServiceInfo() {
  return Promise.all([
    getPageContent("general", "service_day"),
    getPageContent("general", "service_time"),
    getPageContent("general", "service_address")
  ]);
}
```

Why key-value store?

1. **Editable by non-technical users** — No code changes needed
2. **Centralized** — All dynamic content in one table
3. **Cacheable** — Rarely changes; can be cached aggressively
4. **Typed access** — TypeScript ensures correct key names

> **Note:** Role logic lives in `lib/adminRoles.ts` (not `lib/roles.ts`) — five independent per-user booleans stored in the `admin_roles` table, enforced by `middleware.ts` on every `/admin/*` and `/api/admin/*` request. See Authentication & Authorization below for the route → role mapping.

`lib/turnstile.ts`

Cloudflare Turnstile verification, shared by `/login` and Smart Search:

```
export async function verifyTurnstileToken(token: string | null | undefined, ip: string): Promise
// POSTs to challenges.cloudflare.com/turnstile/v0/siteverify with TURNSTILE_SECRET_KEY.
// Fails closed: returns false if the secret isn't set, the token is missing, or the request erro

export function signVerifiedCookie(): string        // `${Date.now()}.${hmac}` using TURNSTILE_CO
export function isVerifiedCookieValid(value): boolean // timing-safe compare + TTL check (30 min)
export const TURNSTILE_SESSION_TTL_MS = 30 * 60 * 1000;
```

Used by: - `app/api/turnstile/verify/route.ts` — verifies a token, sets the `ts_verified` cookie - `app/login/actions.ts` (`adminSignIn`) — verifies the `cf-turnstile-response` form field before attempting sign-in - `app/api/chat/route.ts` — requires `isVerifiedCookieValid(cookies.ts_verified)` before running Smart Search

`lib/smartSearch/tools.ts` — Smart Search tools

```
export interface ToolContext { seenUrls: Set<string> }
export function createToolContext(): ToolContext   // one per /api/chat request

export async function executeTool(name: string, rawArgs: string, ctx: ToolContext): Promise<ToolR
```

- `find_products` , `get_weather` , `get_directions` — unchanged from prior behavior (see Components → `FloatingSmartSearch.tsx`).
- `search_web` — Tavily `/search` with `search_depth: "advanced"` (curated chunks, not raw page tops) and the API key sent as an `Authorization: Bearer` header (not in the body). Snippets truncated to `SNIPPET_CHARS` (1200 chars). Every result URL is recorded into `ctx.seenUrls` .
- `extract_page` — Tavily `/extract` , `extract_depth: "advanced"` . **Only** opens a URL already present in `ctx.seenUrls` for that request — this is the sole guard against the model being prompted (by a visitor or by web content) into fetching an arbitrary URL. Content truncated to `EXTRACT_CHARS` (8000 chars).
- Deliberately does **not** request Tavily's own `include_answer` summary: it was observed conflating Destiny Church Tees Valley with an unrelated Scottish charity of a similar name and reporting a wrong income figure. The model instead reads raw snippets/page content and is told (via `lib/siteKnowledge.ts`) to cross-check the charity/company number before quoting any figure.

`lib/rateLimit.ts`

Per-IP sliding-window rate limiter with escalating cooldowns, **in-memory** (a `Map` , not a database table) — adequate for this site's traffic, per-instance on serverless. Callers namespace keys (e.g. `contact:${ip}`) so endpoints don't trip each other's limits:

```
const WINDOW_MS = 60_000;      // 1-minute sliding window
const MAX_PER_WINDOW = 15;     // requests allowed per window
const COOLDOWN_MS = [5, 10, 20, 40, 60].map((m) => m * 60_000); // escalating cooldowns per offen

const rlStore = new Map<string, { timestamps: number[]; cooldownUntil: number; offenses: number }

export function clientIp(source: { headers: Headers } | Headers): string { /* ... */ }
// checkRateLimit(key) reads/writes rlStore, returns { limited, retryAfterMs }
```

// Used by: // Form submissions (contact, hire, apply for job) // `/api/chat` (Smart Search) — separate simpler per-IP counter, 20 req/min // Login attempts (`lib/loginRateLimit.ts` , a related but distinct limiter)

`lib/training.ts`

Training course management:

```
export interface TrainingCategory {
  id: string;
  name: string;
  description: string;
  icon: string;
  order: number;
}

export interface TrainingModule {
  id: string;
  categoryId: string;
  title: string;
  description: string;
  content: string;  // Markdown or HTML
  duration: number; // Minutes
  videoUrl?: string;
  completed: boolean;
}

export async function getTrainingCategories(): Promise<TrainingCategory[]> {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("training_categories")
    .select("*")
    .order("order");
  return data ?? [];
}

export async function getUserCourseProgress(userId: string) {
  // Fetch completed modules for this user
  // Used to show progress bars and completion badges
}
```

`lib/events.ts` & `lib/events.server.ts`

`lib/events.ts` is client-safe: the `EventCardVariant` union, `parseCardVariant` for the preview routes' `?card=` param, and `RESERVED_EVENT_SLUGS` (currently `{"new"}` , because `app/whats-on/new/` statically shadows `[slug]`).

`lib/events.server.ts` is `server-only` and holds everything touching the feed or Supabase: - `getEventIndex()` — fetch + `buildEventIndex` , `revalidate: 300` - `getFeaturedEvent()` — merges admin overrides over live feed data, applies the promote window and auto-expiry, and handles the feed-outage snapshot fallback - `getActiveEventPopup()` — the same row projected for the popup; returning non-null is what suppresses the generic site popup

`lib/governance.server.ts`

`server-only`. The single place the site talks to either regulator, powering `/governance`.

- `getGovernanceData()` — the only public entry point. Fetches Companies House and the Charity Commission concurrently and returns registration details, officers, filings, trustees, five-year financial history and annual-return submissions, plus a `sources` flag marking each regulator `"live"` or `"fallback"`.
- Exports `CHARITY_NUMBER` (1119951), `COMPANY_NUMBER` (06261423), the two public register URLs, and the `formatRegisterDate` / `formatCurrency` helpers the section components share.

Two rules shape the implementation:

1. **Never look up by name.** Several unrelated organisations are also called "Destiny Church" — notably Scottish charity SC017898, "Destiny Church Trust". Every request is a direct lookup by the hardcoded number, and each response is checked to confirm the number returned matches the number requested. A mismatch is discarded and the section falls back, so a wrong-entity record can never reach the page. The same warning lives in `lib/siteKnowledge.ts` and `lib/smartSearch/tools.ts`.
2. **Never throw.** Same contract as `lib/events.server.ts`: a missing key, timeout, non-OK status or malformed payload resolves to `null`, and the page renders a stored snapshot instead of 500-ing. The two regulators degrade independently — a Companies House outage leaves charity data live.

Auth differs per regulator: Companies House uses HTTP Basic with the API key as the username and a blank password; the Charity Commission uses an `Ocp-Apim-Subscription-Key` header. Charity Commission responses are read through a case- and underscore-insensitive `field()` helper because their payload casing has shifted between endpoint revisions and the full schema sits behind the developer-portal login — a casing change upstream degrades one field rather than the whole section.

Caching is weekly (`revalidate: 604800` on every fetch, matching the page), which keeps both keys far inside their rate limits (Companies House allows 600 requests per 5 minutes).

`lib/jobs.ts` & `lib/hr.ts`

Similar patterns for job listings and HR management.

Authentication & Authorization

Supabase Auth Flow

1. **User signs up/in:** - Email + password sent to Supabase - Returns JWT token - Token stored in secure HTTP-only cookie (via `@supabase/ssr`)
2. **On every request:** - Next.js middleware checks for valid token - If valid, attached to request context - If expired, refresh token is used automatically
3. **On protected routes:** - Server component checks if user exists - Redirects to login if not authenticated
4. **Server actions & API:** - Get user ID from token - Query database with service role (not user role) - Apply RLS policies based on table settings

"Keep me signed in"

The login form (`app/login/LoginClient.tsx`) has a "Keep me signed in" checkbox (`name="remember"` , checked by default). `app/login/actions.ts` 's `adminSignIn` reads it and:

- Passes `{ remember }` into `utils/supabase/server.ts` 's `createClient` , which strips `maxAge` / `expires` from the Supabase auth cookies it writes when `remember` is `false` , turning them into browser-session cookies.
- Sets a small side cookie, `sb-remember` (`utils/supabase/sessionCookie.ts`), recording the choice — `"1"` with a 400-day `maxAge` when remembered, `"0"` as a session cookie when not.

This side cookie exists because `@supabase/ssr` unconditionally re-applies its own 400-day default `maxAge` every time it refreshes the auth cookies — which happens on every `/admin/*` request via `utils/supabase/middleware.ts` . Without it, a "don't remember me" session would silently become persistent the moment middleware refreshed the token. Middleware reads `sb-remember` on each request and re-strips `maxAge` on any cookies it sets when the value is `"0"` . A missing `sb-remember` cookie (e.g. sessions created before this feature) defaults to remembered, so existing sessions aren't unexpectedly downgraded.

`adminSignOut` deletes the `sb-remember` cookie alongside signing out of Supabase.

Authorization Layers

Access levels live in `lib/adminRoles.ts` + the `admin_roles` table — five independent per-user booleans (`training_admin` , `event_admin` , `store_admin` , `site_admin` , `super_admin` ; see admin_roles). Auth *and* role enforcement both happen centrally in `middleware.ts` , not in `app/admin/layout.tsx` (which is a client component purely responsible for the sidebar/header shell; it does not check auth or roles itself).

Route → role mapping (`ROUTE_RULES` in `lib/adminRoles.ts` ; `super_admin` always passes and isn't repeated per rule; anything under `/admin` or `/api/admin` that isn't listed is Super Admin only — fail closed):

Role	Admin pages	API routes
`training_admin`	`/admin/training/**`	`/api/admin/training/**`
`event_admin`	Courses (`alpha` , `recovery` , `bible-course` , `cap-money` , `featured-course`) + Announcements except Banner (`popup` , `featured-event` , `event-popup` , `nfc`)	`/api/admin/{alpha-events,events,featured-course,featured-event,popup,nfc}`
`store_admin`	`/admin/store/**`	`/api/admin/{store,shop-hero}/**`
`site_admin`	`/admin/posts` , `/admin/redirects`	`/api/admin/{posts,redirects}/**`
`super_admin`	Everything, plus Banner, Clear Cache, HR, and `/admin/users`	`/api/admin/{banner,revalidate,hr,users}/**`

`/admin` (dashboard) and `/api/admin/logout` stay open to any authenticated admin. `/admin/users` (Super Admin only, `app/api/admin/users/**`) is where roles are assigned — a tickbox per role per user. `GET /api/admin/me/roles` lets the sidebar (`AdminSidebar.tsx`) know which sections to show; it's a UI convenience only, not an authorization boundary.

Layer 1: Middleware (`middleware.ts`)

```
// Matcher: "/admin/:path*", "/api/admin/:path*" (+ a "/lite" redirect helper, unrelated to auth)
export async function middleware(request: NextRequest) {
  const { supabase, supabaseResponse } = createClient(request);
  const { data: { user } } = await supabase.auth.getUser();

  if (pathname === "/api/admin/logout") return supabaseResponse;      // always reachable
  if (pathname === "/admin/login") return NextResponse.redirect(new URL("/admin", request.url));

  // Public admin auth pages must stay reachable while logged out
  if (["/admin/forgot-password", "/admin/reset-password"].some((p) => pathname.startsWith(p))) {
    return supabaseResponse;
  }

  if (!user && pathname.startsWith("/api/admin")) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }
  if (!user) return NextResponse.redirect(new URL("/login", request.url));

  // Access levels — see lib/adminRoles.ts for the route → role mapping.
  const roles = await getRoles(createServiceClient(), user.id);
  if (!hasAccess(roles, pathname)) {
    if (pathname.startsWith("/api/admin")) {
      return NextResponse.json({ error: "Forbidden" }, { status: 403 });
    }
    return NextResponse.redirect(new URL("/admin?forbidden=1", request.url));
  }

  return supabaseResponse;
}
```

Layer 2: Database (RLS)

```
-- Example: Only service role can read HR staff
CREATE POLICY "service only" ON hr_staff USING (false);
```

API routes use the service role key for reads/writes (RLS is bypassed), relying on the middleware gate above rather than per-table role checks. `admin_roles` itself follows the same deny-all pattern — read only via `createServiceClient()`.

Why two layers? - **Defense in depth** — the middleware gate is the single source of truth for "is this visitor allowed into this `/admin` section"; RLS (deny-all + service role) means even a bug in the app layer can't expose sensitive tables to the anon key. - **Fail closed** — a new admin page that isn't added to `ROUTE_RULES` is Super Admin only by default, rather than accidentally open to everyone.

Layer 0: Cloudflare Turnstile (bot gate, ahead of both layers above)

Two public surfaces are bot-gated with Cloudflare Turnstile before the layers above ever run — the admin login form (protects password-guessing) and the Smart Search chat API (protects the OpenAI/Tavily spend behind it):

- **`/login`** (`LoginClient.tsx` + `app/login/actions.ts`) renders a **visible** Turnstile widget inline in the form. `adminSignIn()` verifies the submitted `cf-turnstile-response` token server-side (`verifyTurnstileToken()`) before even attempting `signInWithPassword`.

- **Smart Search** (`FloatingSmartSearch.tsx`) runs an **invisible** challenge (`size: "invisible"`, `execution: "execute"`) the first time a visitor sends a message in a session, posts the resulting token to `POST /api/turnstile/verify`, and gets back a signed `ts_verified` cookie (HMAC'd, 30-minute TTL — `lib/turnstile.ts`). `POST /api/chat` rejects any request without a valid, unexpired cookie (403), independent of the per-IP rate limit. If the invisible challenge can't silently pass, the widget falls back to rendering a **visible** Turnstile challenge in the chat panel and auto-resends the pending message once solved.
- Both paths share `lib/turnstile.ts`, which fails closed: if `TURNSTILE_SECRET_KEY` or `TURNSTILE_COOKIE_SECRET` isn't configured, verification/cookie-validation always fails rather than silently letting requests through.

Session Management

- **Session storage:** Secure HTTP-only cookie (managed by `@supabase/ssr`)
- **Token refresh:** Automatic when expired (hooks in Providers.tsx)
- **Logout:** Clear cookie, redirect to home
- **CSRF protection:** Next.js built-in (automatic for POST requests)

Configuration

`next.config.ts`

```
const nextConfig: NextConfig = {
  // Include ffmpeg binary for serverless functions (if needed)
  outputFileTracingIncludes: {
    "*": ["node_modules/ffmpeg-static/ffmpeg"],
  },

  // Permanent redirects (handled at edge)
  async redirects() {
    return [
      { source: "/prayer-request", destination: "/connect-card", permanent: true },
      { source: "/prayer-requests", destination: "/connect-card", permanent: true },
      // Retired 2026-08-06; /governance now carries its financial content
      { source: "/annual-report-2025", destination: "/governance", permanent: true }
    ];
  },

  // Security headers
  async headers() {
    return [
      {
        source: "/(.*)",
        headers: [
          {
            key: "Permissions-Policy",
            value: "camera=(), microphone=(), geolocation=()"  // Disable dangerous features
          }
        ]
      },
      // Long-lived cache for static assets (immutable = only update if filename changes)
      {
        source: "/img/:path*",
        headers: [
          { key: "Cache-Control", value: "public, max-age=31536000, immutable" }
        ]
      }
    ];
  },

  // Image optimization
  images: {
    minimumCacheTTL: 2592000, // 30 days
    // Allow images from these domains
    remotePatterns: [
      { protocol: "https", hostname: "i.ytimg.com" },      // YouTube thumbnails
      { protocol: "https", hostname: "storage.buzzsprout.com" },  // Podcast images
      { protocol: "https", hostname: "cdn.churchsuite.com" },     // ChurchSuite events
      { protocol: "https", hostname: "lwmnrbglbtbyypzcenzf.supabase.co" },  // Our Supabase bucke
      // ... more patterns
    ]
  }
};
```

Styling: Tailwind v4, no `tailwind.config.ts`

This project uses **Tailwind CSS v4**, which has no JavaScript config file. Everything lives in `app/globals.css`:

```
@import "tailwindcss";          /* establishes @layer theme, base, components, utilities */

@theme inline {
  --color-destiny-orange: #f58021;
  --color-destiny-orange-dark: #d96d10;
  --color-destiny-red: #fd0000;
  --color-destiny-blue: #0857ba;
  --color-destiny-green: #028002;
  --color-destiny-purple: #8106b1;
  --color-destiny-grey: #363f48;
  --color-destiny-white: #ffffff;
  --color-destiny-black: #000000;
  --color-destiny-brown: #2c1a0e;
  --font-sans: var(--font-roboto), system-ui, -apple-system, sans-serif;
  --font-heading: Arial, "Helvetica Neue", sans-serif;
}
```

Tokens declared in `@theme inline` become utilities automatically, so `--color-destiny-orange` gives you `text-destiny-orange`, `bg-destiny-orange`, `border-destiny-orange` and so on. Add a colour or font by adding a variable here — there is no config file to edit.

Fonts are loaded by `next/font` in `app/layout.tsx` (Roboto for body, Anton for display) and exposed as `--font-roboto` / `--font-anton`.

Cascade layers — read this before adding global CSS

`@import "tailwindcss"` establishes `@layer theme, base, components, utilities`. **Unlayered CSS beats every layered rule regardless of specificity**, so a global style written outside a layer will silently outrank every Tailwind utility, and no amount of specificity in the utility will win.

Two rule sets in `globals.css` were unlayered and caused exactly that:

- `h1, h2, h3 { font-family: var(--font-heading) }` beat every `font-*` utility, so a utility could never change a heading's font. Now in `@layer base`.
- The `.rte-content` prose rules beat every utility used inside rich-text content, which made content blocks impossible to style. Now in `@layer components`.

When adding global CSS, put it in the right layer. Base element defaults go in `@layer base`; reusable classes go in `@layer components`. Leave it unlayered only when you genuinely intend it to beat everything.

Note that `@layer base` still loses to utilities, which is why blocks that want a non-Arial heading set `style={{ fontFamily: FONT_ROBOTO }}` — an inline style beats layered CSS entirely. See `components/blocks/tokens.ts`.

`tsconfig.json`

- `strict: true` — Strict type checking
- `jsx: "react-jsx"` — React 17+ JSX transform
- `@/*` — Path alias for imports (e.g., `@/components/Button`)

`.env.local` Template

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...

# YouTube
YOUTUBE_API_KEY=AIza...
YOUTUBE_CHANNEL_ID=UCxx...
YOUTUBE_CHANNEL_HANDLE=DestinyOnlineChurch       # optional; overrides the CHANNEL_HANDLE constar
YOUTUBE_CHANNEL_VANITY=destinychurchteesvalley   # optional; overrides the CHANNEL_VANITY constar
LIVE_DISABLED=                              # set to 1 to force the /live page and banner off ai

# Email
RESEND_API_KEY=re_...
PAGE_AUDIT_FROM=Destiny AI <noreply@support.squaremediagroup.org>  # from-address for system ale
SMART_SEARCH_ALERT_RECIPIENT=malachi@squaremediagroup.org          # Smart Search down/recovered

# OpenAI (for Smart Search chat — gpt-4.1-mini, tool-calling)
OPENAI_API_KEY=sk-...

# Regulator APIs for /governance (optional — page falls back to a stored snapshot without them)
COMPANIES_HOUSE_API_KEY=                         # HTTP Basic, key as username + blank password
CHARITY_COMMISSION_API_KEY=                      # sent as the Ocp-Apim-Subscription-Key header

# Smart Search tools (optional — each degrades gracefully without its key)
NEXT_PUBLIC_GOOGLE_MAPS_EMBED_API_KEY=AIza...   # get_directions embed
TAVILY_API_KEY=tvly-...                          # search_web + extract_page

# Cloudflare Turnstile (gates /login and /api/chat — see Authentication & Authorization)
NEXT_PUBLIC_TURNSTILE_SITE_KEY=0x...
TURNSTILE_SECRET_KEY=0x...
TURNSTILE_COOKIE_SECRET=                         # random secret for signing the ts_verified cook

# Stripe (shop checkout)
STRIPE_SECRET_KEY=sk_live_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...
SHOP_TEST_BYPASS=            # leave unset in production — enables /api/store/checkout/bypass

# GitHub (CI/CD + "Report a Bug" footer form — used by components/report-bug/actions.ts
# to open issues on SquareMediaGroup/destinychurch via the GitHub REST API)
GITHUB_TOKEN=ghp_...

# Smart Search health cron (GET /api/health/smart-search) — Vercel Cron bearer token
CRON_SECRET=            # if unset, the health check runs unauthenticated (local/dev)

# Feature flags (also toggleable via the `service_status` DB table, e.g. 'smart_search')
ENABLE_SMART_SEARCH=true
```

Deployment & Performance

Deployment Target: Vercel

Why Vercel? - **Next.js first-party** — Built by Vercel team - **Edge functions** — Faster redirects, rate limiting - **ISR (Incremental Static Regeneration)** — Cache pages, revalidate on-demand - **CDN** — Automatic global distribution - **Analytics** — Built-in performance monitoring

Caching Strategy

Content	Strategy	TTL	Invalidation
Static assets (`/img` , `/fonts`)	Immutable	1 year	Filename change
Home page	ISR	1 hour	`revalidatePath("/")`
Sermon archive	ISR	4 hours	`revalidatePath("/sermons")`
Dynamic pages (`/[slug]`)	ISR	24 hours	`revalidatePath("/[slug]")`
Redirects	Edge	Infinite	Deploy
API responses	None	-	Fresh on every request
Images	Browser cache	30 days	`next/image` optimization

Performance Optimizations

1. **Image optimization:** - `next/image` component auto-resizes - WebP format for modern browsers - Lazy loading by default
2. **Code splitting:** - Dynamic imports for heavy components - Suspense boundaries for loading states
3. **Database queries:** - No N+1 queries (use `select()` joins) - Cache at Supabase level (pg_cache extension) - Use materialized views for complex queries
4. **Edge caching:** - Long TTLs on assets - Geography-aware CDN (UK origin)
5. **Monitoring:** - Vercel Analytics — tracks Web Vitals - Vercel Speed Insights — performance timeline - Custom events logged to analytics

---

Key Design Decisions & Rationale

Decision 1: Service Role Key for Backend

Why: Instead of client tokens accessing database directly:

- **Security** — Client can't be trusted; always use server role for writes
- **RLS consistency** — Service role bypasses RLS; we apply auth in app code
- **Error messages** — Can give user-friendly feedback (vs. RLS "you don't have permission")
- **Audit logging** — API routes log who did what; database doesn't

Decision 2: Supabase for Database

Why: Not a custom Node/Express API:

- **PostgreSQL power** — JSON, full-text search, PostGIS for geo (future)
- **RLS built-in** — Security at database layer
- **Real-time** — Subscriptions for live updates (not currently used)
- **Managed** — No DevOps overhead

Decision 3: Server Components Over Client

Why: Use React server components wherever possible:

- **SEO** — Content rendered server-side, searchable
- **Security** — Secrets stay on server (not in browser)
- **Performance** — Less JavaScript shipped to client
- **Simplicity** — No useState/useEffect for data fetching

Decision 4: AI Knowledge Base Over Retrieval

Why: Single `siteKnowledge.ts` instead of RAG:

- **Reliability** — Model grounded in actual facts; no hallucination risk
- **Control** — Easy to edit facts without retraining
- **Cost** — Fewer API calls (no embedding search)
- **Speed** — Lower latency (fewer round-trips)

Decision 5: TipTap for Rich Text

Why: Not Slate, Draft.js, or Lexical:

- **Smaller bundle** — ~50KB gzipped (vs. 150KB+)
- **Extensions** — Underline, highlight, image, YouTube, placeholder
- **Prosemirror foundation** — Battle-tested
- **No config** — Works out of box

File Manifest

Public Pages

- `app/page.tsx` — Home (hero, latest sermon, CTAs)
- `app/sermons/page.tsx` — Sermons (video-first featured message, audio archive)
- `app/sermons/[id]/page.tsx` — Sermon detail (video, next steps)
- `app/live/page.tsx` — Livestream (hero + sections, with a client island for the glass player / off-air card)
- `app/about/page.tsx` — About church
- `app/beliefs/page.tsx` — Statement of faith
- `app/kids/page.tsx` — Kids ministry
- `app/youth/page.tsx` — Youth ministry

- `app/young-adults/page.tsx` — Young adults
- `app/missions/page.tsx` — Missions partners
- `app/serve/page.tsx` — Volunteer opportunities
- `app/connect/page.tsx` — Connect groups
- `app/give/page.tsx` — Giving & donations
- `app/visit/page.tsx` — Plan a visit
- `app/new-here/page.tsx` — First-time visitor guide
- `app/hire/page.tsx` — Venue hire enquiry
- `app/contact/page.tsx` — Contact form
- `app/connect-card/page.tsx` — Prayer requests
- `app/alpha/page.tsx` — Alpha course
- `app/whats-on/page.tsx` — Events (ChurchSuite embed)
- `app/jobs/page.tsx` — Job listings
- `app/jobs/[slug]/page.tsx` — Job detail
- `app/shop/page.tsx` — Store (product grid)
- `app/shop/[slug]/page.tsx` — Product detail (variants, gallery)
- `app/shop/cart/page.tsx` — Shopping cart (Zustand + localStorage)
- `app/shop/checkout/page.tsx` — Stripe Payment Element checkout
- `app/shop/checkout/success/page.tsx` — Order confirmation
- `app/baptism/page.tsx` — Baptism information & registration
- `app/child-dedication/page.tsx` — Child dedication requests
- `app/safeguarding/page.tsx` — Safeguarding & child protection
- `app/governance/page.tsx` — Charity/company registration, trustees, finances & filings (live from both regulators)
- `app/help/page.tsx` — Help centre & FAQ
- `app/training/page.tsx` — /training resource library
- `app/volunteer/page.tsx`, `app/links/page.tsx`, `app/destiny-recovery/page.tsx`, `app/dckids/page.tsx` — Volunteer sign-up, next-steps links, recovery info, kids camp campaign
- `app/accessibility/page.tsx`, `app/privacy/page.tsx`, `app/terms/page.tsx`, `app/data-gdpr/page.tsx` — Preferences & legal pages
- `app/[slug]/page.tsx` — Dynamic catchall (posts table)

Admin Pages

- `app/login/page.tsx` — Staff sign-in (there is no `app/admin/login`; that path is a stale-bookmark redirect to `/admin`)
- `app/admin/page.tsx` — Admin home (dashboard)
- `app/admin/banner/page.tsx` — Banner management
- `app/admin/popup/page.tsx` — Pop-up management
- `app/admin/redirects/page.tsx` — Redirect management
- `app/admin/cache/page.tsx` — Cache invalidation

- `app/admin/alpha/page.tsx`, `app/admin/bible-course/page.tsx`, `app/admin/cap-money/page.tsx`, `app/admin/recovery/page.tsx` — Course event management; four-line wrappers over `components/admin/CourseAdminPage.tsx`
- `app/admin/featured-course/page.tsx` — What's On featured course picker
- `app/admin/store/page.tsx` — Store management
- `app/admin/store/products/new/page.tsx` — Create product
- `app/admin/store/products/[id]/page.tsx` — Edit product (variants, stock)
- `app/admin/store/orders/page.tsx` — Orders list
- `app/admin/store/orders/[id]/page.tsx` — Order detail (fulfillment)
- `app/admin/store/hero/page.tsx` — Shop hero slides (add/edit/reorder rotating hero)

Component Directory

- **~120 components** organized by feature:
- Global: Header, Footer, Providers, CookieBanner, Analytics, FloatingSmartSearch
- Admin: Sidebar, AdminHeader, RichTextEditor (shared editor), Sheet (mobile bottom sheet), blocks/ (palette, inspector, outline, tools), posts/training/hr editors
- Ministry-specific: Kids, Youth, Young Adults, Alpha
- Governance: registration cards, trustees/directors, financial history, filings, source note
- Shop: ProductCard, ShopProductGrid, ShopHero, cart, checkout
- Forms: ConnectCard, ContactForm, HireForm
- Content: Training, Jobs, HR

Libraries (`lib/`)

- **~40 utility files** including:
- Supabase clients (admin, browser)
- API wrappers (YouTube, OpenAI, Resend, Stripe, Companies House + Charity Commission)
- Domain logic (sermons, jobs, HR, training, shop)
- Email templates
- Rate limiting (in-memory), auth is handled by `middleware.ts` (no `roles.ts`)
- Feature flags (`serviceStatus.ts`)

API Routes (`app/api/`)

- **Admin endpoints:** Banners, redirects, pop-ups, cache revalidation, posts, training, alpha-events, featured-course, HR, store management
- **Public endpoints:** `/api/chat` (Smart Search tool-calling chat, Turnstile-gated), `/api/turnstile/verify` (Cloudflare Turnstile token check), YouTube (videos/status/thumbnail/live), Alpha info, training unlock + read timer (`/api/training/posts/[id]/timer`)
- **Store endpoints:** Stripe checkout + Payment Element, order management
- **Webhooks:** Stripe only (`/api/webhooks/stripe`) — no GitHub or Vercel deployment webhook route
- **App BFF (`/api/app/v1/*`):** Backend-for-frontend routes serving the native iOS app. Because the Swift client cannot import `@destiny/shared`, every one of these routes fully normalises its payload server-side (absolute URLs, timezone-qualified timestamps, sanitised HTML, real booleans) and wraps it in a

versioned envelope (`{ status: "ok"|"degraded", generatedAt, data, notice }`) via `lib/appApi.ts` + `lib/appSerializers.ts`. Absolute URLs are stamped from `SITE_ORIGIN`, which resolves in order `NEXT_PUBLIC_SITE_URL` → `VERCEL_PROJECT_PRODUCTION_URL` → the Vercel literal (`https://destinychurch.vercel.app`), so emitted thumbnail/event/ `.ics` links always point at the origin that serves this app rather than the church's `destinytees.uk` nginx site. Seven endpoints:

- `config` — remote config for the More tab (ChurchSuite form links, service times, venue, giving URL, `minSupportedBuild` kill switch); `revalidate = 300`.
- `home` — the whole Home screen in one request (featured event, next service, live status, latest sermon, latest podcast episode) to avoid five cold-launch round trips.
- `events` / `events/[slug]` — the events feed, built with `buildEventIndex` so app and web share the same slugs; the full payload ships in the list response so the detail screen needs no network.
- `sermons` — YouTube sermon videos; a quota trip degrades (with a "Watch on YouTube" link) rather than errors.
- `podcast` — the DCTV Buzzsprout feed (direct-MP3 `audioUrl` for AVPlayer range streaming).
- `live` — live-stream status, polled every 30s, mirroring `contexts/LiveContext.tsx`'s offline grace.
- A degraded response overrides the caller's cache down to `s-maxage=30` so a ChurchSuite blip isn't pinned in the edge cache for the full TTL. Dev-only `?simulate=degraded|empty|error` exercises the app's non-happy states (hard-disabled in production). The legacy `/api/app/events` route is removed.

Mobile App (Native SwiftUI iOS)

The Expo/React Native scaffold was removed in favour of a **native SwiftUI app** (commits `3f68680` onward). The app lives in `mobile/` and is a pure renderer over the `/api/app/v1/*` BFF — it does no feed normalisation of its own, because the BFF already does all of it. - **Project & build.** Swift 6 with `SWIFT_STRICT_CONCURRENCY: complete`, built against the iOS 26 SDK (which opts standard components into Liquid Glass) with an iOS 18 deployment target. The Xcode project is **generated, not committed**: `mobile/project.yml` is the XcodeGen source of truth (sources are globbed, so adding a Swift file needs no project edit), and `mobile/Makefile` wraps the `xcodegen generate` + `xcodebuild` flow (`make project` / `make build` / `make test` / `make launch`). Bundle id `uk.destinytees.app`, portrait-only, iPhone + iPad. `DC_API_BASE` is set per-configuration (`http://localhost:3000` in Debug, `https://destinychurch.vercel.app` in Release). The Release base points at the Vercel origin that actually serves this Next.js app, **not** the church's public `destinytees.uk` domain (currently an nginx site that 404s every `/api` route); it moves back to the public domain only once that domain is pointed at Vercel. - **Structure** (`mobile/DestinyChurch/`): - `App/` — `DestinyChurchApp` entry point and `RootTabView`, the five-tab shell (**Home / Sermons / Events / Give / More**). Live is deliberately *not* a tab — it surfaces as a Home hero card, a Sermons badge, and a More row instead. - `Features/` — one folder per tab (`Home`, `Sermons`, `Events`, `More` incl. `GiveView`), each with its SwiftUI views and an `@Observable` model. - `Networking/` — `APIClient` (the single network entry point; `Endpoint` enum is the only place a path string appears), `Envelope` (decodes the BFF's status/notice wrapper), `APIError`, `LoadState`. - `Models/` — `Codable` mirrors of the BFF payloads (`AppConfig`, `EventSeries`, `Sermon`, `PodcastShow`). - `Design/` — `Brand` (colours), `Typography`, `GlassSurface`, `StateContainer` (loading/empty/error scaffolding), `ComingSoonView`. - `Web/` — `BrowserView`, an in-app browser for ChurchSuite forms opened from the More tab. - `Resources/` — `Info.plist`, `config-fallback.json` (a baked-in copy of `/api/app/v1/config` so the first launch has content before the network returns), and the generated `Assets.xcassets`. - **Generated assets.** `scripts/generate-ios-assets.mjs` (`npm run gen:ios-assets`) rasterises the app icon and launch logo from the brand SVGs in `public/` and emits colorsets from `packages/shared/src/design/tokens.ts`, so the app's colours can never drift from the website's. - **Remote config.** The More tab, giving link, service times and venue all render from `/api/app/v1/config`, so changing a ChurchSuite form slug is a server-side JSON edit, not an App Store release; `minSupportedBuild` in that payload is a kill switch for a bad build. - **`packages/shared/`** (`@destiny/shared`) is an npm workspace of framework-agnostic types/logic shared by the web app and the App BFF (the native app consumes it only indirectly, via the BFF's JSON). It ships raw TypeScript (Next transpiles it via `transpilePackages: ["@destiny/shared"]`). Modules: - `src/churchsuite/events.ts` — `ChurchSuiteEvent`, `deduplicateEvents`, `fetchChurchSuiteEvents`, `churchSuiteEventUrl`, `eventSignupUrl`, `stripEmoji`, `eventDescriptionText`, and `eventImage`. Two feed shapes to be careful with: `images` is an empty **array** when an event has no artwork, and the `thumb/sm/md/lg` entries are *objects* whose URL sits on `.url` (only the `original_*` keys are bare strings) — always go through `eventImage`, which guards both. - `src/churchsuite/dates.ts` — `parseFeedDate` and friends. The feed sends `"2026-07-30 00:00:00"` (a space, not `T`), which Safari parses as Invalid Date, so **every** read of a feed timestamp goes through `parseFeedDate` and that fix lives in one place. `isUpcoming` compares `datetime_end`, so a multiday event already in progress stays visible. - `src/churchsuite/series.ts` — `buildEventIndex`, the single source of truth for event slugs (grid, cards, detail pages, `generateStaticParams`, sitemap and the admin picker all read it). Groups occurrences by `sequence ?? id` — ChurchSuite's `sequence` is the series key, so the seven "Destiny 12:2" rows collapse to one entry. Slugs are assigned in date order, so the soonest event wins a contested name and later claimants take an identifier suffix. - `src/churchsuite/sanitize.ts` — `sanitizeEventHtml`. An allowlist rewriter that drops every attribute except a validated `href`, because the live feed carries `class="branded"` and `style="color:#f9c100"` (invisible on white) written against ChurchSuite's own stylesheet. Dependency-free on purpose: `isomorphic-dompurify` drags jsdom into the server bundle, which is the trap commit `2185023` had to unwind at the 250MB function limit. - `src/churchsuite/ics.ts` — `buildIcs`. Emits local wall-clock times with `TZID=Europe/London` plus a `VTIMEZONE` block rather than converting to UTC, so DST is the calendar client's problem. - `src/design/tokens.ts` — canonical DC brand palette/typography (pillar colours, accent, gradient), matching `app/globals.css`. - **`docs/mobile-app-scope.md`** (+ printable `docs/mobile-app-scope.pdf`) remains the scoping document — BFF architecture, a self-hosted Matrix homeserver for group chat with safeguarding constraints (adult/minor boundary via ChurchSuite DOB data), ChurchSuite API integration, and payments/sermon-feed reuse. It now also includes **Appendix A (ChurchSuite API v2 technical reference)** and **Appendix B (Apple Human Interface Guidelines considerations)**. Phase 1 (the native SwiftUI tab shell over the `/api/app/v1` BFF) is now built; later phases (chat, payments, push) are still planning-only.

Database Migrations

- **39 migration files** defining schema for:
- URL redirects, hidden videos (removed), site content (banners, pop-ups, page_content)
- Event management (Alpha course, Bible Course, CAP Money, Recovery, featured course, featured event)
- HR management (staff, leave, documents, reviews)
- Job listings & applications
- Feature flags, staff login audit
- Training resource library, standalone posts
- Shop (products, variants, orders, hero slides) and RLS/security hardening passes
- NFC tiles (the `/nfc` "digital back of seats" page, incl. event mode) and admin roles

---

Summary

Destiny Church Tees Valley is a **professional, full-stack Next.js application** that combines:

1. **Public-facing website** — Sermons, ministries, contact, events, livestream
2. **Member engagement** — Prayer requests, volunteering, training, small groups
3. **Admin dashboard** — Content management, HR tools, store management
4. **Ecommerce** — Online store with Stripe payments, product variants, order management
5. **Intelligent features** — Smart Search tool-calling chat assistant, live banner management

Architecture: Server-driven React with Supabase PostgreSQL, deployed to Vercel with ISR caching. All sensitive operations use service-role API proxies with explicit auth checks. Code is type-safe (TypeScript), styled with Tailwind CSS, and tested with Playwright E2E.

Key Differentiators: - **No vendor lock-in** — Open-source stack (Next.js, React, Tailwind, PostgreSQL) - **Security first** — RLS, rate limiting, CSRF protection, signed URLs - **AI-ready** — Grounded knowledge base (`siteKnowledge.ts`) powering Smart Search's tool-calling chat (products, weather, directions, web search) - **Accessible** — Semantic HTML, ARIA labels, keyboard nav - **Mobile-first** — Responsive design, tested on real devices - **Performant** — ISR caching, image optimization, edge functions

This repository serves as a **reusable platform** for churches nationwide, licensed through Square Media Group.

Document Version: 1.0.4
Created: June 18, 2026
For: Destiny Church Tees Valley
By: Square Media Group (Malachi malachi@squaremediagroup.org)